



CARRERA DE ESPECIALIZACIÓN EN SISTEMAS EMBEBIDOS

MEMORIA DEL TRABAJO FINAL

Prototipo de dispositivo de geolocalización basado en tecnología GNSS

Autor:

Ing. Luis David Díaz Charris

Director:

Esp. Ing. Santiago Esteva (FIUBA)

Jurados:

Dr. Ing. Javier Jiménez (Universidad de la Costa, CUC)
Dr. Ing. Emiro De La Hoz (Universidad de la Costa, CUC)
Mg. Ing. Jorge Díaz (Universidad de la Costa, CUC)

*Este trabajo fue realizado en la ciudad de Barranquilla, Colombia,
entre marzo de 2023 y octubre de 2024.*

Resumen

En esta memoria se describe el diseño y desarrollo de un prototipo de dispositivo de geolocalización basado en tecnología GNSS. Este permite reportar la posición geográfica de una cava móvil de transporte de alimentos con una exactitud mínima de diez metros. El prototipo implementa un receptor de señales GNSS capaz de recibir simultáneamente señales de los sistemas GPS y BeiDou, además de incorporar comunicación 4G, lo que permite la posibilidad de transmitir datos vía internet a una aplicación web de monitoreo en tiempo real.

Para el desarrollo del presente trabajo, se aplicaron conocimientos en planificación y gestión de proyectos, ingeniería del software para sistemas embebidos, programación de microcontroladores, sistemas operativos en tiempo real, diseño de circuitos impresos y protocolos de comunicación.

Agradecimientos

Agradezco a:

Dios, por acompañarme siempre.

Mi esposa Yuranys, por su aguante para conmigo.

Ariel Lutenberg por la oportunidad, la confianza y sus inacabables ganas de ayudar.

Martín Menéndez, a Santiago Germino y a Ramiro Ghignone por darme un lugar en su grupo.

Santiago Esteva por su guía en este trabajo.

Índice general

Resumen	I
1. Introducción general	1
1.1. Descripción del trabajo	1
1.1.1. Contexto y problema	1
1.1.2. Alternativa de solución	2
1.2. Estado del arte	2
1.2.1. Antecedentes	3
1.3. Objetivos y alcance	4
1.3.1. Objetivo general	4
1.3.2. Objetivos específicos	4
1.3.3. Alcance	4
2. Introducción específica	5
2.1. Requerimientos	5
2.2. Estructura del sistema	6
2.3. Componentes del sistema	6
2.3.1. Microcontrolador ESP32	7
2.3.2. Receptor GNSS	9
2.3.3. Módulo 4G SIM A7670SA	9
Antena Quectel YG0035AA	11
2.3.4. Sistema operativo de tiempo real	12
FreeRTOS™	12
2.3.5. Servicios web para mapas	12
3. Diseño e implementación	15
3.1. Diseño e implementación general del sistema	15
3.1.1. Diseño e implementación del hardware	15
Diseño del hardware	15
Implementación del hardware	16
Placa base del sistema	16
Adaptador para el módulo Quectel L76	18
3.1.2. Diseño e implementación del firmware	20
Diseño del firmware	20
Arquitectura del firmware	20
Implementación del firmware	22
Flujo general del firmware	23
Implementación de las tareas del sistema	26
Otros componentes del firmware	30
3.2. Diseño e implementación de la interfaz web	33
3.2.1. Diseño de la interfaz web	33
3.2.2. Implementación de la interfaz web	34

3.2.3. Metodología de la implementación	34
4. Ensayos y resultados	41
4.1. Resultado final de la integración del hardware	41
4.2. Pruebas funcionales del hardware	43
4.2.1. Metodología para las pruebas del hardware	43
4.2.2. Resultados y análisis	44
De las pruebas sobre la PCB	44
De las pruebas entre módulos	45
4.3. Pruebas funcionales de la aplicación web	53
4.3.1. Prueba de integración con el hardware	53
Metodología	53
Resultados y análisis	53
4.3.2. Prueba de exactitud de la ubicación	54
Metodología	54
Resultados y análisis	54
5. Conclusiones	57
5.1. Conclusiones generales	57
5.2. Próximos pasos	58
A. Esquemático de la placa base del sistema	59
Bibliografía	61

Índice de figuras

1.1. Ejemplo de una cava móvil ¹	1
2.1. Diagrama general del sistema.	6
2.2. Placa de desarrollo ESP32-Devkit-V1 ²	8
2.3. Módulo Quectel L76 ³	9
2.4. Módulo SIM A7670SA ⁴	10
2.5. Módulo A7670SA ⁵	11
2.6. Antena Quectel YG0035AA ⁶	11
3.1. Arquitectura del hardware desarrollado.	16
3.2. Modelo 2D de la placa base del sistema.	17
3.3. Modelo 3D de la placa base del sistema.	17
3.4. Placa base del sistema versión de una capa (frente).	18
3.5. Placa base del sistema versión de una capa (reverso).	18
3.6. Esquemático del adaptador L76.	19
3.7. Modelo 2D del adaptador L76.	19
3.8. Modelo 3D del adaptador L76.	19
3.9. Adaptador L76 real.	20
3.10. Arquitectura del firmware.	21
3.11. Diagrama de flujo del firmware (parte 1).	24
3.12. Diagrama de flujo del firmware (parte 2).	25
3.13. Diagrama de flujo del firmware (parte 3).	26
3.14. Arquitectura MQTT ⁷	34
3.15. Maqueta de la interfaz web.	37
3.16. Versión final de la interfaz web.	38
4.1. Resultado final del hardware desarrollado (exterior).	41
4.2. Resultado final del hardware desarrollado (interior).	42
4.3. Vista real de los componentes del hardware (exterior).	42
4.4. Vista real de los componentes del hardware (interior).	43
4.5. Medición de la salida en el circuito de regulación.	44
4.6. Gráfico de la muestras tomadas en la medición de corriente.	45
4.7. Datos más representativos en la medición de corriente.	45
4.8. Diagrama de conexiones para la prueba del módulo Quectel L76.	46
4.9. Conexiones reales para la prueba del módulo Quectel L76.	46
4.10. Vista de las tramas NMEA en la terminal <i>Hercules</i>	48
4.11. Diagrama de conexiones para la prueba del módulo Quectel L76.	48
4.12. Conexiones reales para la prueba del módulo Quectel L76.	49
4.13. Vista de las tramas AT en la terminal <i>Hercules</i>	50
4.14. Vista de las tramas AT en la terminal <i>Hercules</i>	51
4.15. Fotos de la prueba de la pantalla LCD.	53
4.16. Captura del mapa para la prueba de integración con el hardware.	54

Índice de tablas

2.1. Tabla comparativa de microcontroladores.	7
2.2. Tabla comparativa de API's de mapas.	13
4.1. Mediciones de la exactitud en la geolocalización	55

Dedicado a...

Mis hijos, Tomás y María Helena.

Mi esposa Yuranys.

Mi madre Luz Fabiola.

Capítulo 1

Introducción general

En este capítulo se exponen los aspectos fundamentales del trabajo, con el propósito de familiarizar al lector con las estrategias de desarrollo que se detallan en los capítulos subsiguientes.

1.1. Descripción del trabajo

A continuación, se presenta una descripción del contexto, problema y propuesta de solución que guiaron el presente trabajo.

1.1.1. Contexto y problema

Una cadena de supermercados en Colombia tiene importantes retos logísticos a la hora de abastecer las tiendas distribuidas en todas las ciudades. La empresa implementa centros de abastecimiento que cubren distintas ciudades en determinados rangos de acción. Uno de los retos más relevantes a los que se enfrenta, es la distribución de alimentos en cadena de frío. Dicha tarea, se lleva a cabo a través de cavas móviles que tienen la capacidad de conservar la temperatura interna y refrigerar su interior. Estas cavas representan un activo costoso y son totalmente reutilizables. En la figura 1.1, se puede apreciar un ejemplo de cava móvil para transporte de alimentos.



FIGURA 1.1. Ejemplo de una cava móvil¹.

¹<https://www.coldtainer.es>

El centro de despacho enfrenta problemas logísticos cuando se queda sin cavas disponibles para la distribución. En esta situación, es necesario realizar un esfuerzo adicional para ubicar y recuperar las cavas. La dificultad radica en que no es posible saber su ubicación exacta en tiempo real. Por lo tanto, se requiere solicitar a cada sucursal el reporte de cavas disponibles para su retorno al centro de distribución.

1.1.2. Alternativa de solución

Por lo anterior, se planteó como una alternativa de solución, el desarrollo de un dispositivo que permita rastrear la ubicación de cada una de las cavas. Este dispositivo permite replicarse con facilidad, es de bajo consumo de energía y tiene una exactitud de diez metros en el reporte de la localización.

1.2. Estado del arte

Los sistemas de navegación por satélite son actualmente las herramientas por excelencia en las aplicaciones de movilidad y transporte. La geolocalización es una utilidad necesaria en todos los tipos de transporte: navegación espacial, la aviación, la navegación marítima, los ferrocarriles y el transporte automotor por carretera [1].

Uno de los sistemas de navegación más conocidos es el Sistema de Posicionamiento Global (GPS), desarrollado por el Departamento de Defensa de Estados Unidos. Sin embargo, a nivel mundial existen varios de estos sistemas, y al conjunto de ellos se le conoce como Sistemas de Navegación Global por Satélite (GNSS, del inglés Global Navigation Satellite Systems) [2]. Los GNSS están compuestos por constelaciones de satélites que orbitan la Tierra y transmiten datos sobre su ubicación espacial y temporal. Además, incluyen redes de estaciones de control terrestres y receptores que calculan las posiciones en tierra mediante la técnica de trilateración [2].

Algunos de los GNSS que existen actualmente son [1]:

- El Sistema GPS (Global Position System): de EE. UU., con 30 satélites, a una altura de 20 200 km.
- El Sistema Glonass (Global Orbiting Navigation Satellite System): de Rusia, con 24 satélites, a una altura de 25 500 km.
- El Sistema Galileo (ESA): de Europa, con 30 satélites, a una altura de 23 200 km.
- El Sistema BeiDou (BDS): de China, con 34 satélites, a una altura entre 21 600 km. y 35 800 km.

En la literatura científica, estos sistemas son ampliamente estudiados y revisados, debido a su gran rango de aplicaciones en diferentes áreas del conocimiento (aparte de la navegación) y la técnica. Entre las más relevantes se destacan:

- Ingeniería Civil y Arquitectura. Por ejemplo, para el replanteo de infraestructuras y estructuras, control del movimiento de tierras, medición de perfiles transversales, entre otras.

- Cartografía y Topografía, para mejorar la precisión de la cartografía y la modelización del mundo físico, desde montañas y ríos, hasta calles, edificios, cables y tuberías de los servicios públicos.
- Geodesia y Geofísica, para mejorar la precisión en el estudio de los fenómenos físicos que afectan la forma y dimensiones de la Tierra.
- Agricultura, en el desarrollo de mejores técnicas para la gestión de los cultivos.

1.2.1. Antecedentes

A continuación, se presentan algunos antecedentes en la literatura científica sobre el estudio de los sistemas GNSS.

Di Garzia et al. [3] evaluaron el rendimiento del chip receptor de GNSS TeseoV STA8135GA, de STMicroelectronics. Su trabajo se contextualiza en las aplicaciones automotrices de alta precisión. Determinaron la viabilidad técnica y las ventajas de que en un solo chip se pudiera incorporar la capacidad de recibir señales de tres bandas. Lo anterior, permitió agregar resiliencia y redundancia en caso de fallas completas de alguna de las bandas de frecuencia.

Shu et al. [4] evaluaron el rendimiento de un GNSS multi-constelación para averiguar el impacto en la determinación de la posición en un ambiente dinámico. Lo anterior, para saber cuánto mejora el GNSS la exactitud de la posición. Con su estudio determinaron que:

- En un caso estático, los sistemas GPS, GLONASS y BeiDou tienen el mismo rendimiento.
- En el caso dinámico, los sistemas GPS y GLONASS tienen rendimientos muy similares, donde el de GPS es superior.
- Cuando se combinan GPS, BeiDou, GLONASS y Galileo, la respuesta es significativamente mejor en comparación con el empleo de un único sistema de posicionamiento.

Por su parte, *Villien y Denis* [5] emplean una combinación de tres estrategias para mejorar la calidad de la recepción GNSS:

1. Utilizan un receptor GNSS de múltiples constelaciones y para ampliar el rango de transmisores satelitales de los que obtener información de posicionamiento.
2. Utilizan una antena de banda ultra ancha (UWB, del inglés *Ultra-Wideband*) para optimizar en una única antena la capacidad de recibir señales GNSS con un ancho de banda amplio.
3. Emplean una unidad de medida inercial junto con un filtro Kalman. Esta configuración permite rastrear de forma completa el objeto observado y reducir la carga computacional del sistema.

El trabajo de *Villien y Denis* logra una exactitud en la geolocalización de entre 3 cm y 11 cm en entornos controlados. Además, se evidencia una exactitud horizontal en las mediciones GNSS superior a 40 cm durante una prueba de campo, incluso en condiciones degradadas de recepción de la señal GNSS. Este trabajo es

especialmente relevante en aplicaciones de geolocalización en espacios cerrados o túneles, donde un único sistema GNSS resulta insuficiente.

1.3. Objetivos y alcance

1.3.1. Objetivo general

Desarrollar un prototipo de dispositivo de geolocalización basado en tecnología GNSS, que permita reportar y hacer seguimiento en tiempo real de la posición geográfica de una cava móvil de transporte de alimentos con una exactitud mínima de diez metros.

1.3.2. Objetivos específicos

1. Diseñar la arquitectura de hardware y firmware, del dispositivo de geolocalización con tecnología GNSS.
2. Implementar el hardware y el firmware del dispositivo de geolocalización.
3. Implementar un servicio online para el seguimiento de la posición geográfica.
4. Realizar pruebas de consumo energético, exactitud y funcionalidad del prototipo.

1.3.3. Alcance

Este trabajo abordó:

- El análisis de métodos ingenieriles adecuados y tecnologías disponibles para la implementación del sistema.
- El diseño y prototipado del sistema y subsistemas de hardware requerido para la interconexión con los sistemas GNSS y la red celular.
- El diseño y prototipado del sistema y subsistemas de hardware requerido para la interfaz humano-máquina tales como pilotos, teclas o pantallas.
- El desarrollo del firmware necesario para ejecutar las funciones de conectividad, funciones lógicas y reporte de estados.
- Configuración de la API web necesaria para realizar pruebas de seguimiento en tiempo real del dispositivo.
- Optimización del consumo de energía a través de firmware.

Este trabajo no incluyó:

- El desarrollo de una interfaz web a medida.
- El diseño del aspecto físico del dispositivo.

Capítulo 2

Introducción específica

Este capítulo presenta en detalle los temas aplicados en el desarrollo del trabajo y las definiciones realizadas en el inicio del trabajo. Permite conocer los requisitos y organización de tareas planteados.

2.1. Requerimientos

A continuación, se presentan los requerimientos planteados en la planificación del trabajo:

1. Requerimientos funcionales

- a) El sistema a desarrollar debe ser un prototipo funcional para pruebas en ambiente de laboratorio.
- b) El sistema debe reportar en una plataforma de software la posición geográfica de las cavas en tiempo real.
- c) La plataforma de software deberá permitir la visualización en un mapa de la posición geográfica de las cavas.
- d) La exactitud del reporte de la posición geográfica debe ser de diez metros (10 m).
- e) Como requisito mínimo, el sistema debe ser capaz de conectarse con dos constelaciones de satélites para obtener la posición geográfica.
- f) El hardware debe tener botones para que los operarios puedan reportar la ocupación o desocupación de las cavas.
- g) El hardware debe poseer un display LCD que permita a los operarios visualizar las coordenadas geográficas reportadas por el sistema y el estado de ocupación de la cava.
- h) El hardware debe poseer pilotos que permitan a los usuarios visualizar e interpretar de forma lógica el estado de ocupación de las cavas.

2. Requerimientos no funcionales

- a) El sistema debe ser escalable y debe poderse replicar.
- b) El sistema a desarrollar debe usar componentes electrónicos disponibles en Colombia.

2.2. Estructura del sistema

En la figura 2.1 se observa el diagrama general del prototipo que se construyó. Allí se representa al microcontrolador, que funge como unidad de procesamiento central. Además, se distinguen los componentes de comunicación satelital y celular: un receptor GNSS y un módulo 4G LTE. Se aprecian, las infraestructuras de comunicación que se emplearon para hacer posible la comunicación. Se distingue la infaltable fuente de alimentación para garantizar la energía. Por último, el sistema comprende de una interfaz web como herramienta para la presentación y seguimiento de la información.

En el capítulo siguiente se aborda con más detalle la implementación de cada uno de estos módulos.

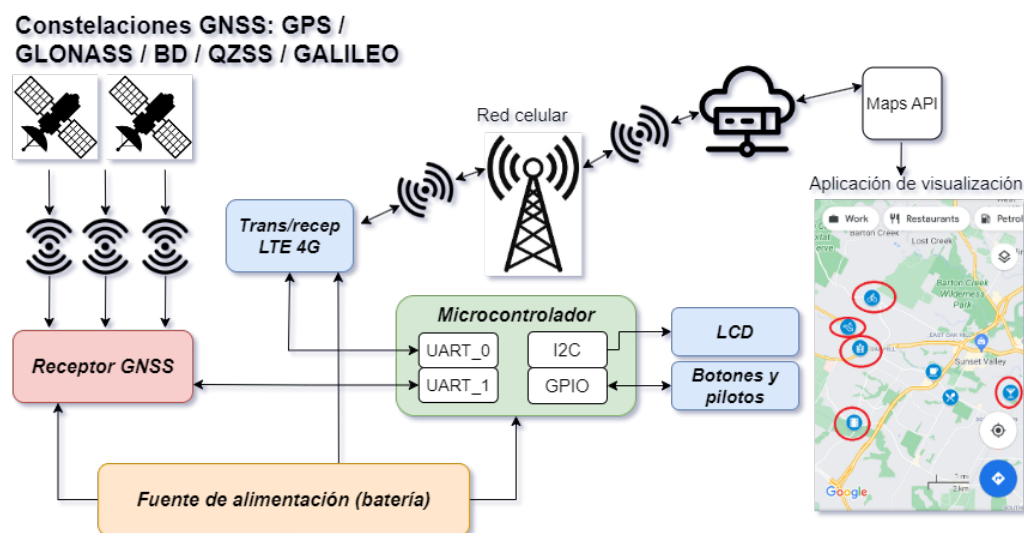


FIGURA 2.1. Diagrama general del sistema.

2.3. Componentes del sistema

En este trabajo se desarrolló un prototipo para realizar el posicionamiento geográfico de cavas móviles de transporte de alimentos. Dicho sistema comprende los siguientes componentes:

- Microcontrolador, que realiza las labores de unidad central de procesamiento.
- Receptor GNSS, para la recepción de datos de posicionamiento.
- Módulo de comunicación 4G, para la conexión a internet.
- Plataforma de software para la presentación de la información.
- Fuente de alimentación.

Para la selección de los principales componentes se buscaron alternativas en el mercado a partir de los requisitos planteados en la sección 2.1.

2.3.1. Microcontrolador ESP32

Luego de realizar un estudio comparativo entre tres plataformas de microcontroladores disponibles en el mercado colombiano, se seleccionó el microcontrolador ESP32. En la tabla 2.1 se presenta un extracto de la comparación entre las plataformas.

TABLA 2.1. Tabla comparativa de microcontroladores.

Característica	ESP-WRoom-32	Arduino ME-GA	STM32 F401
CPU	Dual-core Tensilica Xtensa LX6	ATmega2560 (8-bit)	ARM Cortex-M4 @ 84 MHz
Número de núcleos	2	1	1
Frecuencia del procesador (MHz)	160	16	84
ROM	448 kB	256 kB (Flash)	512 kB (Flash)
RAM	520 kB	8 kB (SRAM)	96 kB (SRAM)
Comunicación	Wi-Fi, Bluetooth, 2x UART, 1x SPI, 1x I2C y 1x USB.	1x USB, 2x UART, 1x SPI e 1x I2C	3x USART, 4x SPI, 3x I2C, 1x CAN y 1x USB
Lenguajes de programación	Arduino, C/C++ y Micropython	Arduino y C/C++	Arduino y C/C++
Cantidad de GPIO's	36	54 (Digital), 16 (Analog)	82
Soporte de hardware para RTOS	Sí	No 16 (Analog)	Sí

El microcontrolador ESP32 comprende una familia o serie de microcontroladores tipo System on Chip (SoC), desarrollado por la empresa Espressif [6].

El ESP32 es un sistema de doble núcleo con dos CPU Harvard Architecture Xtensa LX6. Toda la memoria integrada, la memoria externa y los periféricos se encuentran en el bus de datos y/o el bus de instrucciones de estas CPU. Las dos CPU se denominan PRO_CPU y APP_CPU (de protocolo y aplicación), sin embargo, para la mayoría de los propósitos las dos CPU son intercambiables [6].

Entre las principales características de este microcontrolador se encuentran:

- Espacio de direcciones de 4 GB (32 bits) para el bus de datos y el bus de instrucciones.
- Espacio de direcciones DMA de 328 kB. Trece de sus módulos son capaces de operar con DMA.
- Espacio de direcciones periférico de 512 kB.
- ROM interna de 448 kB.
- SRAM interna de 520 kB.
- Admite hasta 16 MB de Flash SPI o hasta 8 MB de SPI SRAM.

En la figura 2.2 se presenta la placa de desarrollo ESP32-Devkit-V4.

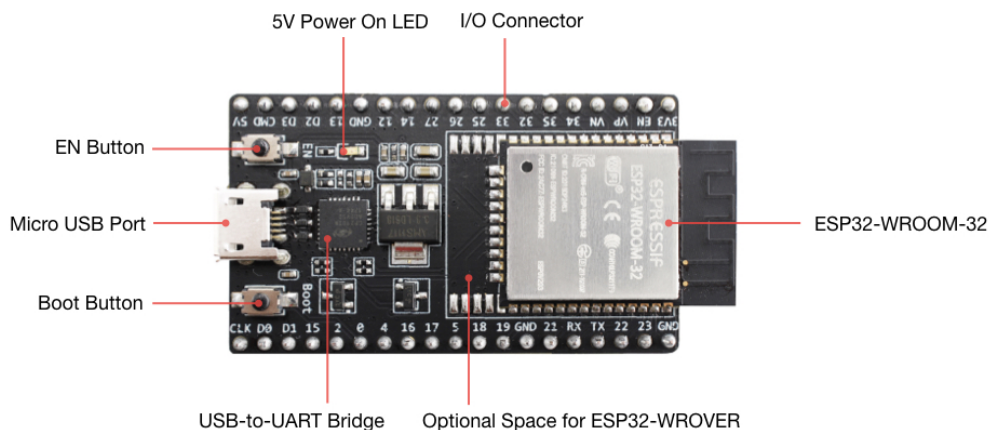


FIGURA 2.2. Placa de desarrollo ESP32-Devkit-V1¹.

A continuación, se presentan los beneficios de haber aplicado el microcontrolador ESP32 en el presente trabajo:

- Es de resaltar la significativa capacidad de cómputo inherente a su diseño, ya que cuenta con un procesador de doble núcleo. Esto fue especialmente útil para implementar multitarea a través de FreeRTOS.
- Incorpora dos módulos de comunicación inalámbrica: Wi-Fi y Bluetooth. Esto permite la conexión con internet y la conexión en redes de área personal.
- Otro punto a favor importante para los requerimientos del trabajo, es que presenta modos de bajo consumo energético configurables.
- El fabricante proporciona una documentación extensa de su producto. Además, provee el *framework* ESP-IDF que es bastante potente y está disponible gratuitamente.
- Debido a que es un producto *Open Source* y *Open Hardware*, goza de una comunidad de desarrolladores bastante activa y una gran disponibilidad de documentación. Estas dos características posibilitan que cualquier inconveniente que se presente tenga una alta probabilidad de ser resuelto.
- Por último, es un dispositivo de bajo costo, con un precio asequible y una relación calidad/prestaciones muy buena. Esto hizo posible abaratar los costos del trabajo.

¹Imagen tomada de: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/esp32/get-started-devkitc.html>

2.3.2. Receptor GNSS

Un receptor de Sistemas de Navegación Global por Satélite (GNSS, por sus siglas en inglés) es un dispositivo que permite realizar posicionamiento global. Esto lo hace a través de la recepción de señales provenientes de distintas constelaciones de satélites. Estas señales contienen información detallada sobre la posición de un objetivo a nivel mundial, lo que ofrece precisiones que abarcan desde los 10 metros hasta los 0,1 milímetros.

Para este trabajo se seleccionó el módulo Quectel L76 (QL76), que es un receptor GNSS que admite cinco sistemas de navegación y posicionamiento global: GPS, GLONASS, Galileo, BeiDou y QZSS. En la figura 2.3 se puede observar el aspecto físico de dicho módulo [7]. Se eligió este debido a la disponibilidad en el mercado local y a que sus características cumplen perfectamente con los requerimientos planteados.

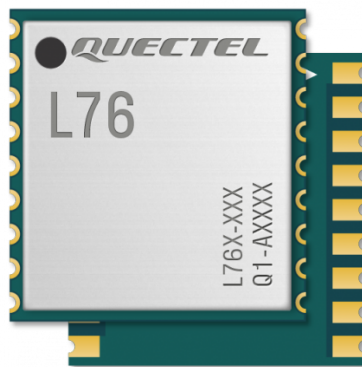


FIGURA 2.3. Módulo Quectel L76².

A continuación, se describen las características principales del módulo QL76 [7]:

- La configuración de constelación predeterminada es GPS+GLONASS.
- Admite interfaces de comunicación UART e I2C.
- Ofrece diversos modos de funcionamiento que permiten la optimización del consumo de energía, a saber: GLP, AlwaysLocate™, Standby y Backup
- Tiene un consumo de energía extremadamente bajo.

2.3.3. Módulo 4G SIM A7670SA

Un transmisor/receptor 4G es un dispositivo terminal que permite la comunicación inalámbrica para dispositivos electrónicos a través de las redes de telefonía móvil de cuarta generación. Estos módulos facilitan la transmisión de datos, mensajes SMS, llamadas y conexión a internet a través de la infraestructura de redes móviles.

Para este trabajo se escogió el SoC (*Sistem on Chip*) 4G SIMA7670SA con tecnología 4G. Soporta las bandas de frecuencia GSM y LTE-FDD. Está optimizado para

²Imagen tomada de: <https://www.quectel.com/product/gnss-l76>

casos de uso de IoT, diseñando bajo la especificación CAT-1. Puede operar en condiciones difíciles, entre las que se pueden mencionar:

- Cobertura en lugares remotos.
- Interferencia por el fenómeno de *handover*.
- Congestión de la red.
- Condiciones meteorológicas adversas.

El SoC también integra el *stack* de protocolos TCP/IP, MQTT. Este se escogió debido a que presenta buenas prestaciones en un tamaño compacto [8].



FIGURA 2.4. Módulo SIM A7670SA³.

Las características principales de este SoC son:

- Frecuencias de trabajo: LTE-FDD(B1/B2/B3/B4/B5/B7/B8/B28/B66), GSM (850/900/1800/1900MHz).
- Soporta interfaces UART, USB, I2C y GPIO.
- Velocidad de descarga/carga en modo LTE de 10/5 Mbps.
- Velocidad de descarga/carga en modo GPRS/EDGE de 236.8/236.8 Kbps.
- Consumo de corriente en modo LTE 3,8 mA.
- Consumo de corriente en modo GSM 3,5 mA.

Para el presente trabajo se utilizó el módulo A7670SA 4G LTE CAT1 MULTIBANDA GSM GPRS IOT (ver figura 2.5) comercial basado en el SoC SIM A7670SA, debido a la facilidad para integrarlo al sistema general. Además, dispone de las interfaces de hardware necesarias para su interconexión con la ESP32, incluye su propio circuito de alimentación y un diseño fiable para evitar las interferencias electromagnéticas.

³Imagen tomada de: <https://www.simcom.com/product/A7670X.html>

⁴Imagen tomada de: <https://ssdielect.com/gsm-y-gprs-1/4872-a7670sa-sin-gps.html>

FIGURA 2.5. Módulo A7670SA⁴.

Antena Quectel YG0035AA

Debido al requerimiento de conectarse, como mínimo, con dos constelaciones de satélites para obtener la posición geográfica, se debe utilizar una antena capaz de captar las frecuencias de transmisión de al menos dos constelaciones satelitales. Para este proyecto, se considera la antena Quectel YG0035AA. En la figura 2.6 se observa la antena.

La antena está diseñada para la recepción de señales de la constelación GPS y BeiDou (BD) y presenta las siguientes características eléctricas:

- Antena tipo activa.
- Rango de frecuencias de 1561 MHz a 1575 MHz.
- Impedancia de entrada de 50 Ohm
- Ganancia entre 0,5 dBi y 1,4 dBi
- Polarización tipo RHCP.

FIGURA 2.6. Antena Quectel YG0035AA⁵.

⁵Imagen tomada de: <https://www.quectel.com/product/yg0035aa-gnss-active-magnetic-mount-antenna/>

2.3.4. Sistema operativo de tiempo real

Un sistema operativo de tiempo real (RTOS, del inglés: *Real-Time Operating System*), es un tipo de sistema operativo diseñado específicamente para controlar y gestionar tareas de tiempo real en sistemas embebidos, como microcontroladores y sistemas integrados. La característica principal de un RTOS es que puede responder a eventos y ejecutar tareas de manera predecible y en un intervalo de tiempo garantizado.

El firmware que se implementó en este trabajo está basado en FreeRTOS™. En el apartado siguiente, se explica qué es y cuáles son las ventajas que se obtuvieron con su utilización.

FreeRTOS™

FreeRTOS™ es un sistema operativo de tiempo real de código abierto diseñado para sistemas embebidos y microcontroladores. Es apto para microcontroladores y microprocesadores pequeños [9]. Las ventajas de haber implementado este sistema operativo en el trabajo son las siguientes:

- Debido a que proporciona un kernel multitarea, permitió generar concurrencia entre tareas, como lo fueron la recepción y parseo de tramas GNSS, la transmisión de datos en tiempo real.
- Gracias a la temporización precisa, se pudo asegurar la transmisión de datos de posicionamiento en tiempo real.
- Permitted la gestión de la memoria y los periféricos UART del microcontrolador. De hecho, con FreeRTOS™, es posible gestionar todos los recursos del dispositivo con colas y semáforos. Al mismo tiempo, el kernel no representa una carga pesada para la capacidad de cómputo del microcontrolador utilizado.
- Se pudo acceder a las ventajas de un RTOS de forma libre con la licencia MIT de FreeRTOS™.
- El código desarrollado es altamente portable, lo que permitirá escalar o transferir fácilmente el prototipo hacia otras plataformas.

2.3.5. Servicios web para mapas

Un servicio de mapas es un conjunto de herramientas y servicios de software que permiten que los mapas estén disponibles en la web.

Para este trabajo se creó una aplicación web basada en los servicios de la biblioteca para JavaScript Leaflet. Esta es de código abierto y ofrece todas las funciones cartográficas necesarias para la creación de aplicaciones que requieren el uso de mapas, como es el caso de la geolocalización.

En la tabla 2.2 se presenta una comparación entre tres bibliotecas que prestan herramientas para la creación de mapas en la web:

TABLA 2.2. Tabla comparativa de API's de mapas.

Característica	Google API	Maps	Leaflet.js	Carto.js
Licencia	De pago, con un nivel gratuito limitado.		Open-source (BSD 2-clause License).	De pago, con un nivel gratuito limitado.
Facilidad de uso	Alta		Alta	Media
Documentación	Extensa y detallada.		Buena, pero poco extensa. Complementa con muchos ejemplos.	Buena, orientada a usuarios con conocimientos en GIS
Soporte y comunidad	Amplio soporte y gran comunidad.		Gran comunidad open-source.	Comunidad activa, pero reducida.
Capacidades de mapa	Incluye vistas de satélite, terreno, y Street View.		Básico, pero extensible con plugins.	Enfocado en visualización de datos geoespaciales.
Personalización	Alta, mediante opciones y controles avanzados.		Muy alta, altamente personalizable con plugins.	Alta, especialmente en visualización de datos.
Rendimiento	Muy bueno, optimizado por Google.		Muy bueno, especialmente en navegadores modernos.	Bueno, puede ser pesado con grandes conjuntos de datos.
Integración de datos	Fácil integración con servicios de Google como Places, Routes, etc.		Necesita plugins adicionales para integración avanzada de datos.	Muy bueno para integración de datos geoespaciales y análisis.
Soporte para móviles	Excelente, completamente responsive.		Muy bueno, compatible con dispositivos móviles.	Bueno, pero depende del tamaño de los datos.
Características avanzadas	Geocoding, Directions, Distance Matrix, Places API, etc.		Depende de plugins externos.	Análisis espacial avanzado, mapas temáticos, SQL API.
Facilidad de integración	Muy fácil con otros servicios de Google.		Fácil con bibliotecas JavaScript estándar.	Requiere conocimiento de GIS y SQL para máxima utilidad.

Capítulo 3

Diseño e implementación

Este capítulo se realiza la descripción detallada de cada módulo de hardware y software desarrollados en el trabajo. Se aplican las definiciones y criterios descritos en capítulos anteriores y se comprende el diseño final del prototipo.

3.1. Diseño e implementación general del sistema

El diseño e implementación general del sistema se subdividió en 3 partes:

1. Del hardware.
2. Del firmware.
3. De la interfaz web.

A continuación, se describen cada una de estas partes.

3.1.1. Diseño e implementación del hardware

En esta sección, se describen las consideraciones de diseño y la implementación del hardware del sistema.

Diseño del hardware

En la figura 3.1 se presenta la arquitectura del hardware desarrollado. Se incluyen todos los componentes descritos en las secciones anteriores.

El sistema cuenta con dos antenas correspondientes a la comunicación GNSS y 4G.

Como parte de la interfaz humano-máquina, se cuenta con una pantalla LCD de 16x2, dos botones para la indicación manual de estado ocupado/desocupado y dos pilotos que permiten visualizar el estado de ocupación actual.

En el diagrama se señalan los protocolos UART e I2C, para la comunicación intercircuitado, que son:

- UART: para la comunicación entre el módulo 4G SIM A7670SA por el puerto UART_0 y el módulo GNSS Quectel L76 por el puerto UART_1.
- I2C: para la comunicación con la pantalla LCD.

Se presenta una fuente de alimentación correspondiente a una batería de 12 VDC. Debido a que los componentes del sistema operan con tensiones diferentes, se

hizo necesaria la implementación de circuitos adicionales. En el diagrama, estos circuitos se denominan «Fuente». Estos suplen dos niveles de tensión distintos:

- 3,3 VDC para el Quectel L76.
- 5 VDC para la ESP32, el módulo SIM A7670SA y la pantalla LCD.

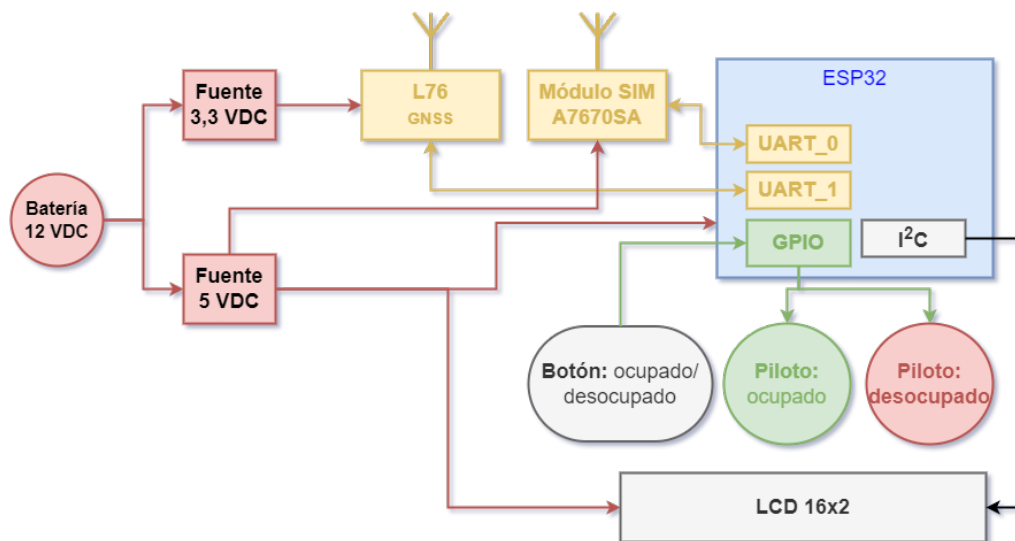


FIGURA 3.1. Arquitectura del hardware desarrollado.

Implementación del hardware

En la etapa de implementación del hardware, se trabajó en dos placas de circuito impreso.

1. Placa base del sistema.
2. Adaptador para el módulo Quectel L76.

Placa base del sistema

La placa base se encarga de interconectar los módulos y periféricos principales del sistema. En el apéndice A se presenta el circuito esquemático correspondiente a esta implementación. En circuito se empleó el regulador MIC29302WU [10], con una configuración de ajuste por defecto en 5 VDC, para la alimentación de la ESP32, del módulo SIM A7670SA, la placa adaptador para el módulo Quectel L76 y la pantalla LCD. En la figura 3.2 se muestra el diseño final del PCB en un modelo de dos dimensiones. Además, en la figura 3.3 se observa el modelo en tres dimensiones.

Los componentes U1, U2 y H3, corresponden a las ranuras donde se conectan la ESP32 DevKit V4, el adaptador para el módulo Quectel L76, y el módulo SIM A7670SA, respectivamente. Las terminales marcadas con CN6 y CN7 permiten la conexión de los pilotos que indican la ocupación de la cava. Las terminales CN4 y CN5 corresponden a las conexiones de los botones de que dispondrá el operario de la cava para la indicación de la ocupación. Por otro lado, las terminales CN2 y CN3 corresponden a la interfaz I2C y la alimentación de la pantalla LCD, respectivamente. La terminal CN1 corresponde a la entrada de alimentación del sistema. Finalmente, las terminales CN9 y CN11 externalizan los pines GPI 34 y

GPI 35, y GPIO 5 y GPIO 18 de la ESP32, respectivamente; esto para facilitar la agregación de nuevas funcionalidades en una etapa posterior.

En las figuras 3.2 y 3.3, se puede ver la versión de dos capas de la placa base. Sin embargo, por razones de tiempo, se decidió implementar una versión de una sola capa que fue manufacturada con una máquina de fresado automático o CNC (*Computer Numerical Control*). En las figuras 3.4 y 3.5, se muestra el resultado final de la manufactura de la placa.

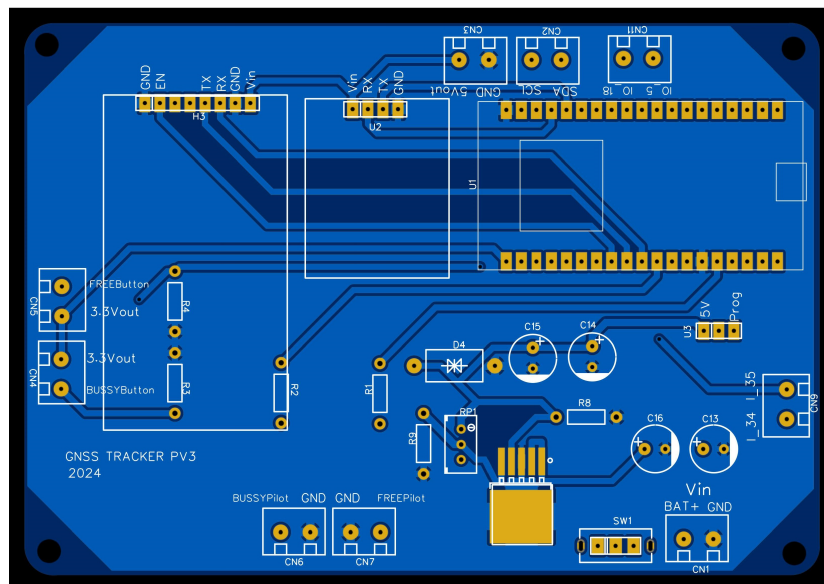


FIGURA 3.2. Modelo 2D de la placa base del sistema.

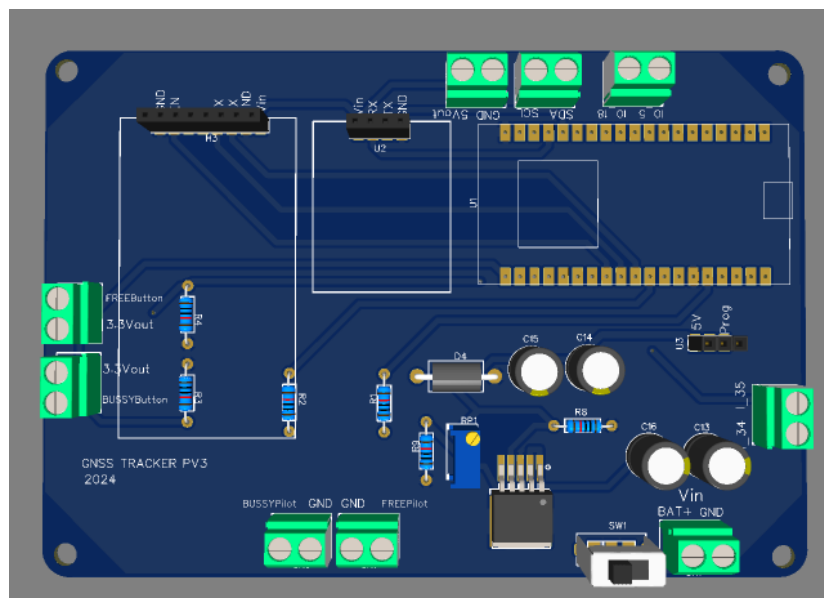


FIGURA 3.3. Modelo 3D de la placa base del sistema.

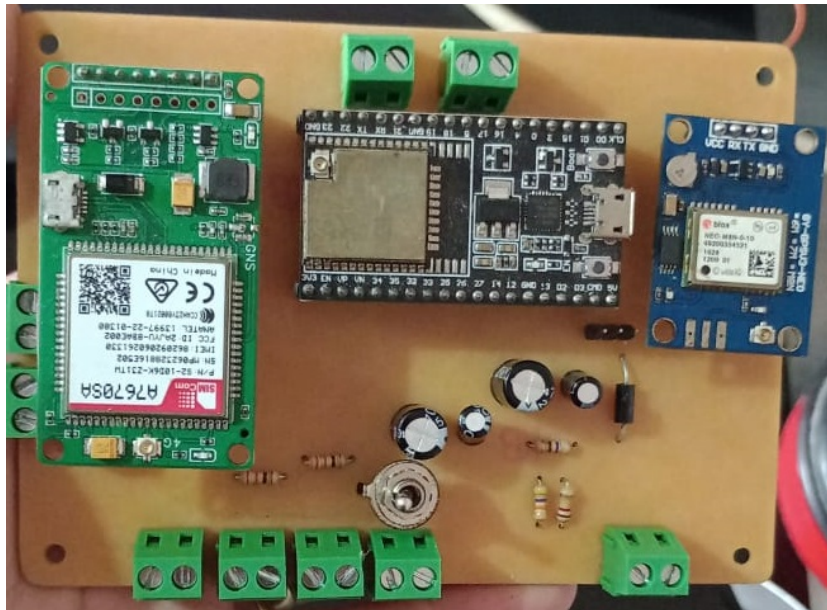


FIGURA 3.4. Placa base del sistema versión de una capa (frente).

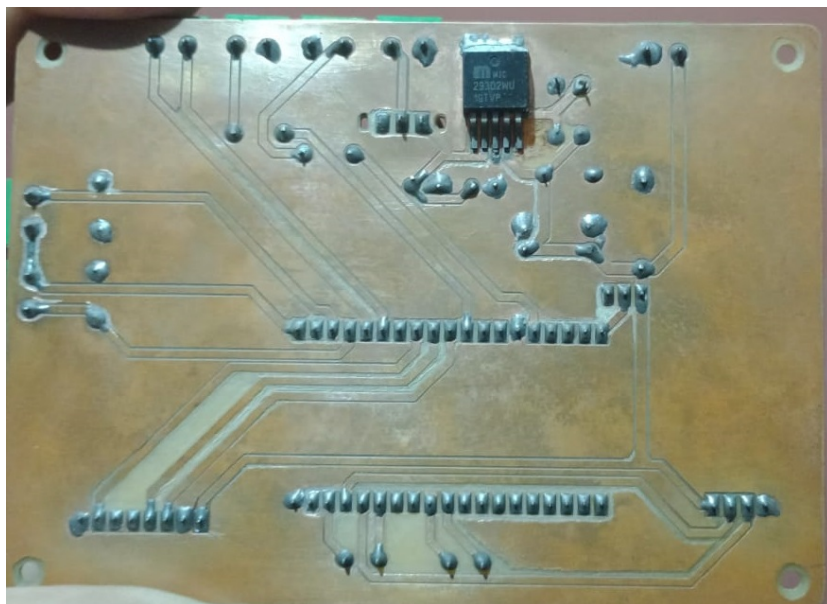


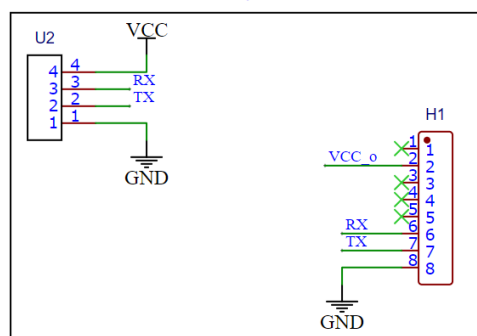
FIGURA 3.5. Placa base del sistema versión de una capa (reverso).

Adaptador para el módulo Quectel L76

El circuito esquemático del adaptador L76 se encuentra en la figura 3.6. En este esquemático se observa la fuente de 3,3 VDC de entrada de alimentación, la conexión UART y la conexión entre el módulo y el microcontrolador ESP32.

En las figuras 3.7 y 3.8, se muestran los modelos 2D y 3D del circuito diseñado. Además, en la figura 3.9 se presenta el resultado final de la manufactura de la PCB.

Conexión entre módulos y entrada de alimentación



Fuente de 3,3 VDC

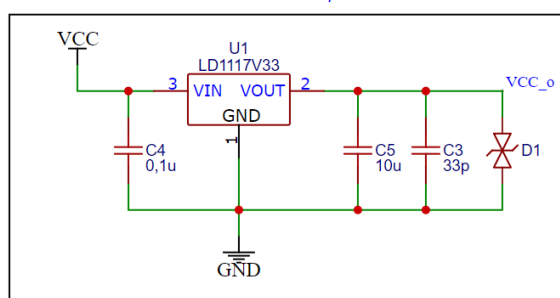


FIGURA 3.6. Esquemático del adaptador L76.

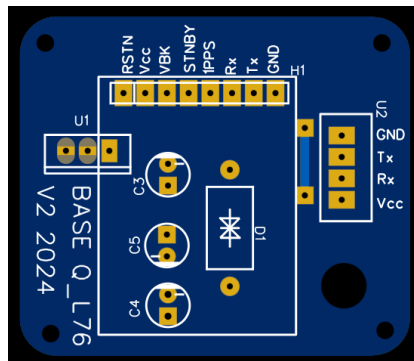


FIGURA 3.7. Modelo 2D del adaptador L76.

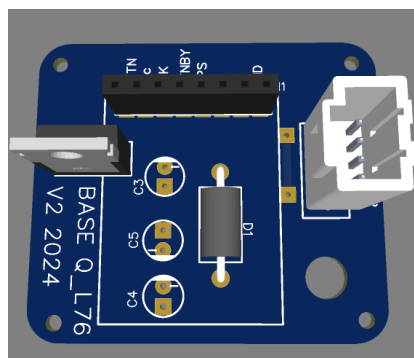


FIGURA 3.8. Modelo 3D del adaptador L76.

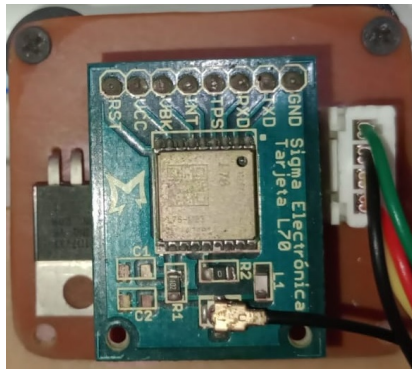


FIGURA 3.9. Adaptador L76 real.

3.1.2. Diseño e implementación del firmware

En esta sección se presentan las consideraciones de diseño y el proceso de implementación del firmware.

Diseño del firmware

El propósito del firmware desarrollado es satisfacer los requerimientos de conectividad, administración del hardware e interfaz con el usuario. A continuación se describen las principales funcionalidades del firmware:

1. Comunicación por comandos AT-UART con el módulo SIM A7670SA, para realizar las funciones de conexión a Internet a través de la red celular, que asegure la conexión con el *broker* MQTT.
2. Comunicación a través de tramas NMEA por el puerto UART con el módulo Quectel L76. Para la recepción e interpretación de los datos de geolocalización de las constelaciones GPS y BeiDou.
3. Monitoreo de los botones de ocupación a través de interrupciones y visualización de los estados de ocupación a través de los pilotos.
4. Generación de interfaz visual a través de la comunicación I2C con la pantalla LCD 16x2.
5. Administración de la energía del sistema a través de los modos de bajo consumo.
6. Administración de los recursos del microcontrolador a través de la implementación del sistema operativo en tiempo real FreeRTOS.

Arquitectura del firmware

De acuerdo con lo planteado anteriormente, en la figura 3.10 se presenta un diagrama de componentes UML, que representa la arquitectura del sistema general, con énfasis en los componentes del firmware.

En este diagrama (figura 3.10) se representan como subsistemas (*Subsystem*) los componentes de hardware como la ESP32, la pantalla LCD, el módulo 4G SIM A7670SA, los botones y los pilotos. Así mismo, se entienden como subsistemas los componentes de firmware proporcionados por el fabricante del microcontrolador, que corresponden a los drivers GPIO, UART y el bus I2C. Además, se hizo

necesaria la implementación de una biblioteca propia para el control de la pantalla LCD a través del bus I2C.

De acuerdo con esto, los componentes del firmware se establecen en la ESP32, que corresponde al dispositivo programable encargado de realizar el control general del sistema.

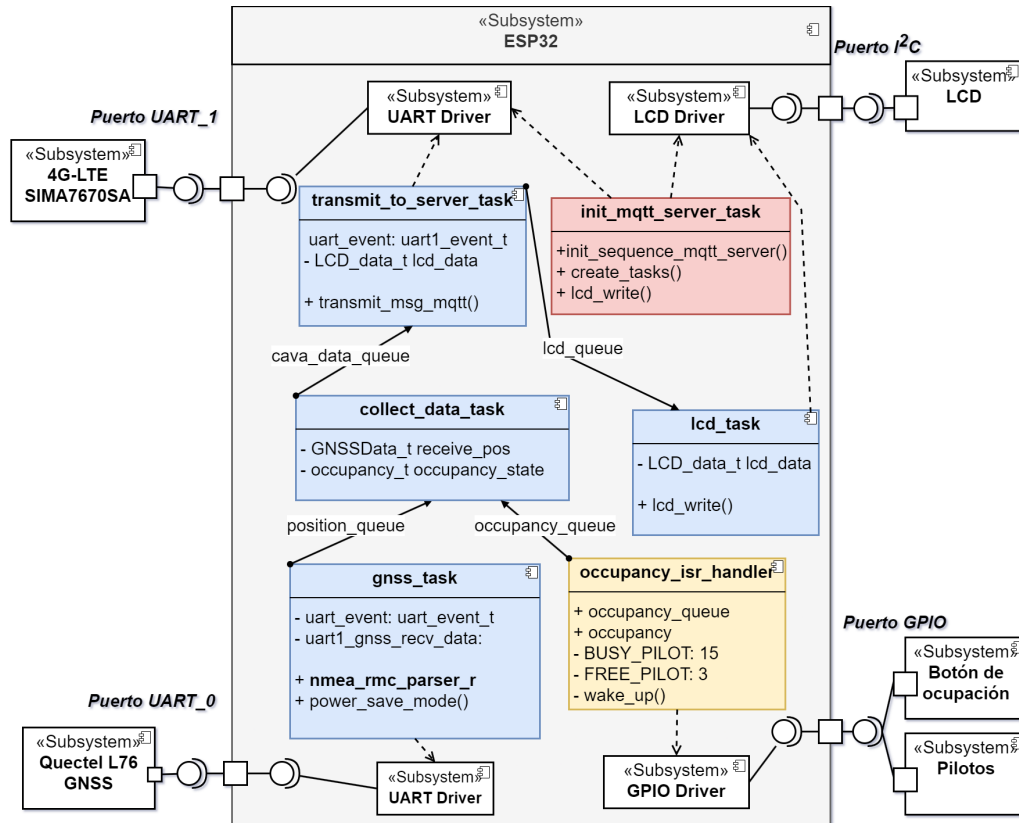


FIGURA 3.10. Arquitectura del firmware.

En el diagrama de la figura 3.10 se presentan los componentes de firmware más relevantes del sistema y la forma en que interactúan entre ellos. A continuación, se describe la funcionalidad de cada componente del firmware:

1. El firmware inicia con la tarea `init_mqtt_server_task()`, resaltada con color rojo, que se encarga de tres funciones primordiales: realizar la inicialización de los servicios MQTT que proporciona el módulo 4G (SIM A7670 SA), realizar la conexión con el *broker* MQTT remoto y, si todo sale bien, desencadena la creación de las demás tareas del sistema. Esta tarea utiliza líneas de comandos AT para comunicarse con el módulo 4G.
2. Tarea `collect_data_task()` se encarga de recibir, a través de colas, la información relacionada con la ocupación y la posición de la cava. Dicha información proveniente de la tarea `gnss_task()` y del manejador de interrupciones `occupancy_isr_handler()`, respectivamente. Luego transmite a la tarea `transmit_to_server_task()` la información recibida, a través de la cola `cava_data_queue`.

3. Manejador de interrupción `occupancy_isr_handler()`, resaltada con color amarillo, es la función que se ejecuta siempre que ocurra una interrupción debida a los botones de ocupación de la cava, a saber, ocupado-desocupado. Este manejador de interrupción detecta qué botón se presionó y comunica a la tarea `collect_data_task()` el estado de ocupación, a través de la cola `occupancy_queue`. Además, esta tarea genera una acción de *wakeup* del sistema cuando se encuentra en modo ahorro de energía.
4. Tarea `gnss_task()`, se encarga primordialmente de detectar la recepción de una trama NMEA del tipo GNRMC, lo que asegura que la posición ha sido calculada con señales de dos o más constelaciones satelitales; en este caso, GPS y BeiDou. La detección de tramas se realiza a través de interrupciones del módulo UART de la ESP32. Esta tarea descarta todas las tramas que no sean compatibles con el estándar NMEA y todas las que no sean del tipo GNRMC. Además, se encarga de la extracción de la información de las tramas RMC a través de la función `nmea_rmc_parser_r()`. Esta última es una implementación de tipo *thread-safe* para asegurar la compatibilidad con FreeRTOS. Además, comunica la posición actual a la tarea `collect_data_task()`, a través de la cola `position_queue`. Por último, esta tarea verifica si la posición no ha cambiado en los últimos 5 minutos, en caso de no cambiar, colocará al sistema completo en modo ahorro de energía.
5. Tarea `transmit_to_server_task()`, se encarga de ejecutar la secuencia de transmisión de datos a través del protocolo MQTT, con el módulo 4G. La comunicación entre microcontrolador y el módulo se realiza a través de comandos AT. Esta tarea recibe los datos provenientes de la tarea `collect_data_task()`, los transmite hacia el *broker* MQTT en la nube con la función `transmit_msg_mqtt()`. Al finalizar, construye un paquete de información con los datos de ocupación, posición y resultado de la transmisión MQTT, para ser transferido a través de la cola `lcd_queue` hacia la tarea `lcd_task()`.
6. Tarea `lcd_task()` se encarga de administrar adecuadamente el acceso a los recursos de la pantalla LCD, para evitar sobrescritura o pérdida de información relevante que deba ser mostrada a través de la pantalla LCD.

Implementación del firmware

El firmware se implementó en el *framework* ESP-IDF desarrollado por la empresa ESPRESSIF, para las series de SoCs ESP32, ESP32-S y ESP32-C. Este *framework* proporciona un SDK apto para el desarrollo de cualquier aplicación genérica en estas plataformas, a partir de lenguajes de programación C y C++. Además, está optimizado para aplicaciones de Internet de las Cosas (IoT, del inglés *Internet of Things*).

También se utilizó el entorno de desarrollo (IDE) *PlatformIO*, que se ejecuta como una extensión nativa en el IDE de *Visual Studio Code*. *PlatformIO* permite usar todas las funcionalidades de ESP-IDF de una manera fiable y ágil, sin enmascarar funcionalidades o configuraciones avanzadas del *framework*. Además, permite ejecutar las versiones más estables de ESP-IDF, proporciona soporte por la comunidad de desarrolladores, es multiplataforma y de software libre.

Para la implementación del firmware se utilizó el lenguaje de programación C y se implementó el sistema operativo en tiempo real FreeRTOS. Una ventaja importante que ofrece ESP-IDF es que está desarrollado sobre FreeRTOS, por tanto, el firmware se ejecuta nativamente en este.

Por petición del cliente, la codificación correspondiente al firmware no podrá ser mostrada, sin embargo, se empleará pseudocódigo para sustentarla y no comprometer la calidad del trabajo desarrollado.

Flujo general del firmware

El firmware tiene la capacidad de ejecutar multitarea, gracias a que se realizó una implementación con FreeRTOS. El flujo general del programa se muestra en las figuras 3.11, 3.12 y 3.13. El programa se dividió en tareas, que son funciones puntuales que ejecutan una acción concreta y colaboran entre sí para operar el sistema en su conjunto. Las diferentes tareas se ejecutan de forma independiente y existe un flujo de información que se sucede de una tarea a otra. Como mecanismo de comunicación entre tareas, se utilizan las colas. Este comportamiento del firmware es relevante desde el punto de vista en que el sistema puede estar recibiendo, procesando y transmitiendo datos de posicionamiento de manera casi concurrente.

En el diagrama de la figura 3.11, se muestra la siguiente secuencia de acciones:

1. Inicialización de todos los periféricos del hardware que son requeridos para operar con los módulos externos. Tal es el caso de los periféricos de comunicaciones UART e I2C. El periférico UART es utilizado para la comunicación entre los módulos SIM A7670SA [8] (comunicación celular, sección 2.3.3) y el módulo Quectel L76 (receptor GNSS, sección 2.3.2).
2. Inicialización de los periféricos GPIO e instala el servicio de interrupciones, para preparar la captura de los estados de ocupación que se da a través de botones.
3. Inicialización del bus I2C que es requerida para la comunicación con la pantalla LCD. Luego de esto, ocurre la inicialización de la pantalla LCD, para mostrar al usuario toda la información del inicio del sistema.
4. Inicialización de las colas, que son los canales de comunicación entre las distintas tareas del programa y se crea la tarea `init_mqtt_server_task()`, que se explica a continuación.

En el mismo diagrama de la figura 3.11 (bifurcación A), se muestra el flujo de la tarea `init_mqtt_server_task()`, que realiza lo siguiente:

1. Inicializar los servicios MQTT en el módulo SIM A7670SA, para luego iniciar la conexión con el *broker* MQTT en la nube.
2. Si lo anterior ocurre sin problemas, se crean las demás tareas del sistema: `gnss_task()`, `collect_data_task()`, `transmit_server_task()` y `lcd_task()`.
3. El *scheduler* de FreeRTOS empieza a gestionar la ejecución de las tareas creadas previamente y cambia entre ellas según su prioridad y el tiempo asignado. El código principal (la función `main()`) ya no continúa su ejecución. Este punto se representa con la bifurcación B.

4. Para finalizar, la tarea `init_mqtt_server_task()` se elimina a sí misma con la función de FreeRTOS `vTaskDelete()`. Esto, con el objetivo de liberar recursos del microcontrolador.

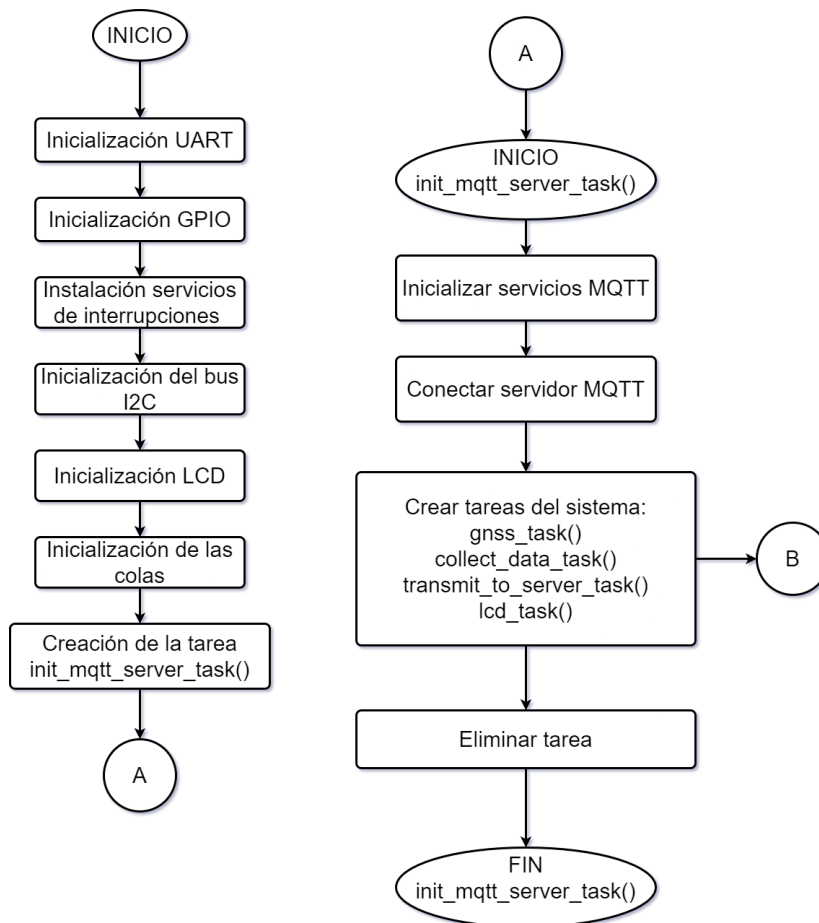


FIGURA 3.11. Diagrama de flujo del firmware (parte 1).

En la figura 3.12 se representan dos tareas:

1. Tarea `gnss_task()`, procesa datos GNSS capturados por `UART_0`, envía las posiciones procesadas a `collect_data_task()` a través de la cola `position_queue` y verifica si la posición ha cambiado en un intervalo de 5 minutos para entrar en modo de ahorro de energía si no hay cambios.
2. Tarea `collect_data_task()`, monitorea eventos de ocupación y posición, guarda los datos de ocupación y posición cuando ocurren eventos y envía los datos de ocupación a `transmit_server_task()` a través de la cola `cava_data_queue` para su transmisión al servidor.

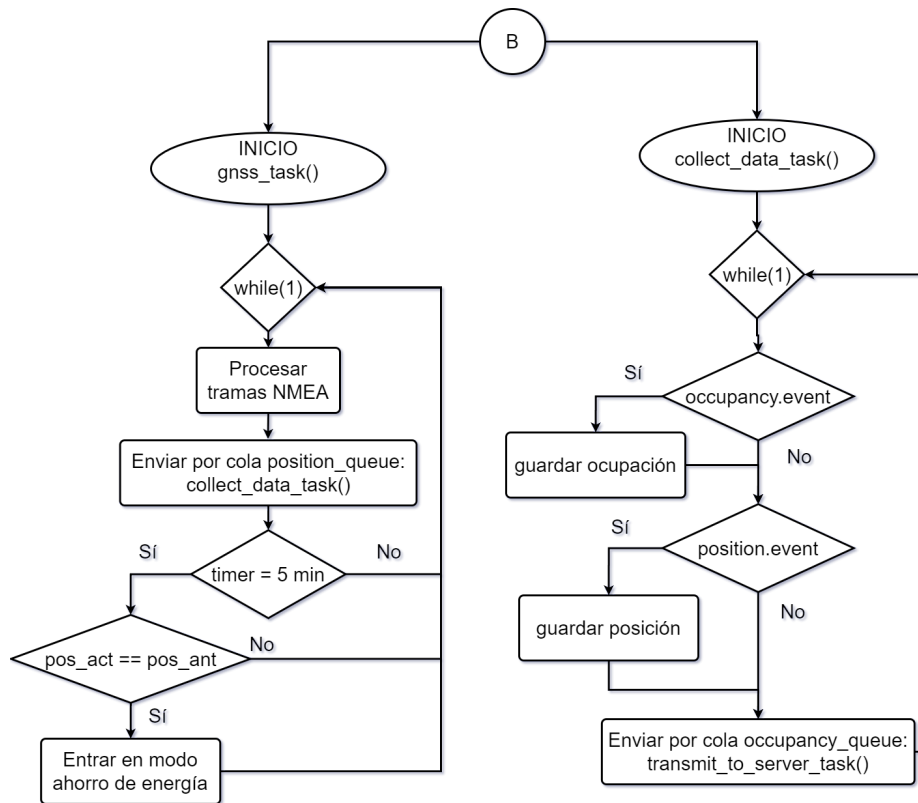


FIGURA 3.12. Diagrama de flujo del firmware (parte 2).

En la figura 3.13 se representan dos tareas:

1. Tarea `transmit_to_server_task()` verifica continuamente si hay eventos de datos de la cava, transmite los datos de la cava al servidor MQTT cuando hay un evento y envía los datos a la cola `lcd_queue` para ser procesados por `lcd_task()`.
2. Tarea `lcd_task()` verifica continuamente si hay eventos en la cola `lcd_queue` y muestra la información de posición, ocupación y estado del servidor en la pantalla LCD cuando hay un evento.

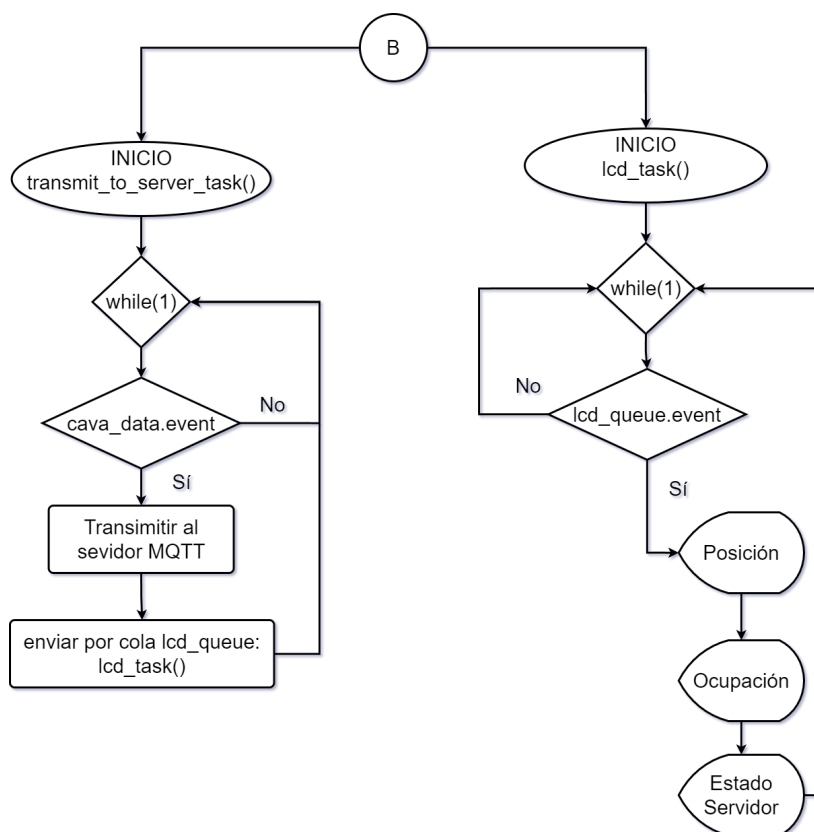


FIGURA 3.13. Diagrama de flujo del firmware (parte 3).

Implementación de las tareas del sistema

A continuación, se explica en forma de pseudocódigo la implementación de cada una de las tareas del sistema.

En el código 3.1, se muestra el pseudocódigo que refleja la implementación de la tarea `init_mqtt_server_task()`. La tarea espera a que el módulo SIMA7670 se encuentre listo para la comunicación por comandos AT, para esto, en la línea 3 se implementa un bucle que bloquea la tarea mientras espera el estado “READY” del módulo. Luego, en la línea 5, se inicializan los servicios necesarios para operar con MQTT. Esto incluye la configuración de parámetros y la preparación de las conexiones. Después, en la línea 6 se realiza la conexión al servidor MQTT; esto asegura que el sistema pueda enviar y recibir mensajes a través de este protocolo. Si la conexión es exitosa, se continúa con la creación de las tareas del sistema (ver línea 9). Después de crear y poner en marcha las tareas del sistema, en la línea 14, se elimina la tarea `init_mqtt_server_task()`, lo que implica la liberación de recursos. Si hay una falla en la conexión con el servidor, se realizarán un total de 3 intentos de conexión, además del intento inicial (ver línea 6). Si falla la conexión se reporta el problema por la LCD con la instrucción de la línea 17 y se procede a la eliminación de la tarea en la línea 18.


```

1  init_mqtt_server_task()
2  {
3      while ("READY" == SIM_A7670SA)
4          Esperar
5      Inicializar servicios MQTT en SIM_A7670SA
6      while (4 == intentos)
7          Conectar con broker MQTT en la nube
8          if ("OK" == Conexion_Servidor)
9              Crear tareas:
10                 gnss_task()
11                 collect_data_task()
12                 transmit_to_server_task()
13                 lcd_task()
14             Eliminar: init_mqtt_server_task
15             break
16         else
17             Reportar por LCD: Falla conexion broker
18             Eliminar: init_mqtt_server_task
19     }

```

CÓDIGO 3.1. Pseudocódigo de la tarea `init_mqtt_server_task()`.

En el código 3.2, se presenta el pseudocódigo de la tarea `gnss_task()`. La tarea espera cada evento del puerto `UART_0` (ver línea 4) para procesar las tramas NMEA con datos provenientes del receptor GNSS, lo que se puede ver en la línea 5. Luego del procesamiento, envía la información extraída por la cola `position_queue`, que es gestionada por la tarea `collect_data_task()` (ver línea 7). En la línea 8, el sistema verifica si ha pasado un intervalo de tiempo de cinco minutos para comprobar tres cosas:

- Sí: si se alcanzaron los cinco minutos y las posiciones son iguales, el sistema entra en modo de ahorro de energía.
- No: si no se han alcanzado los cinco minutos y las posiciones son diferentes, el sistema vuelve al inicio del bucle.
- No: si no han pasado cinco minutos, el sistema vuelve al inicio del bucle.

```

1  gnss_task()
2  {
3      while (1)
4          if (DATA == evento_UART_0)
5              Procesar trama:
6                  posicion_actual =
9                      ↪ nmea_rmc_parser_r()
7              Enviar por cola position_queue:
9                  ↪ posicion_actual
8              if (Temporizar 5)
9                  if (posicion_anterior ==
10                      ↪ posicion_actual)
11                      sleep_mode:
12                          ↪ Modulo_celular
13                          sleep_mode: Modulo_GNSS
14                          sleep_mode: ESP32
15
16              posicion_anterior = posicion_actual
17  }

```

CÓDIGO 3.2. Pseudocódigo de la tarea gnss_task().

En el código 3.3, se presenta el pseudocódigo del manejador de interrupciones `occupancy_isr_handler()`, que aunque no es una tarea mapeada en el *scheduler* de FreeRTOS, cumple un papel primordial, puesto que permite capturar los eventos de los botones de ocupación del sistema. El manejador está asociado con la subrutina de interrupciones que se debe ejecutar cuando ocurre un evento de GPIO. Esta se activa cuando se presiona un botón y se captura cuál botón ha sido presionado (ver línea 4). Si el botón presionado es el de “libre” se actualiza el estado para indicar que el recurso está libre y se envía un estado “libre” a través de `occupancy_queue`, como se muestra en las líneas 5 a 7. Si el botón presionado es el de “ocupado”, se actualiza el estado para indicar que la cava está ocupada y se envía un estado “ocupado” a través de `occupancy_queue`, según las instrucciones de las líneas 9 a 11.

```

1  occupancy_isr_handler()
2  {
3      Capturar boton presionado
4      switch (bonton)
5          case boton_libre:
6              Setear los pilotos como libre
7              Enviar por cola occupancy_queue: "Libre"
8
9          case boton_ocupado:
10              Setear los pilotos como ocupado
11              Enviar por cola occupancy_queue: "
12                  ↪ Ocupado"
13  }

```

CÓDIGO 3.3. Pseudocódigo de la función `occupancy_isr_handler()`.

En el código 3.4, se presenta el pseudocódigo de la tarea `collect_data_task()`.

A través de la línea 4, la tarea verifica si hay un evento de ocupación, si este ocurre, se guarda la información correspondiente en una estructura local. También verifica si hay un evento de posición (ver línea 7), si ocurre, se guarda la información correspondiente en la estructura local. Finalmente, en la línea 10, la información de ocupación se envía por la cola `cava_data_queue` que es gestionada por la tarea `transmit_to_server_task()`.

```

1 collect_data_task()
2 {
3     while (1)
4         if (DATA == occupancy_queue.event)
5             datos_cava.occupancy = occupancy
6
7         if (DATA == position_queue.event)
8             datos_cava.position = position
9
10        Enviar por cola cava_data_queue: datos_cava
11    }

```

CÓDIGO 3.4. Pseudocódigo de la tarea `collect_data_task()`.

En el código 3.5, se muestra la tarea `transmit_to_server_task()`. La tarea verifica si hay un evento relacionado con los datos de la cava (ver línea 5). Si no hay evento, el sistema vuelve al inicio del bucle; si hay un evento, los datos de la cava se transmiten al servidor MQTT y luego a la tarea `lcd_task()` + el resultado de la transmisión, como se muestra en las líneas 6 a 8.

```

1 transmit_to_server_task()
2 {
3     posicion_anterior
4     while (1)
5         if (DATA == cava_data_queue.event)
6             Transmitir al broker MQTT:
7                 transmission_state =
8                     Enviar por cola lcd_queue: cava_data+
9     }

```

CÓDIGO 3.5. Pseudocódigo de la tarea `transmit_to_server_task()`.

En el código 3.6, se presenta el pseudocódigo de la tarea `lcd_task()`. Esta tarea verifica si hay un evento en la cola (ver línea 4), si no hay evento, el sistema vuelve al inicio del bucle; si hay un evento, se muestra la siguiente información en la pantalla LCD:

- La posición geográfica (líneas 5 a 8).
- El estado de ocupación (línea 9).
- El estado de la conexión servidor MQTT (línea 10).

```

1  lcd_task()
2  {
3      while (1)
4          if (DATA == lcd_queue.event)
5              Escribir_LCD: position.lat
6              Escribir_LCD: position.long
7              Escribir_LCD: position.time
8              Escribir_LCD: position.val // Validez de
           ↪ la posicion
9              Escribir_LCD: occupancy
10             Escribir_LCD: server_state
11 }

```

CÓDIGO 3.6. Pseudocódigo de la tarea lcd_task().

Otros componentes del firmware

Además de las tareas y funciones descritas en la sección 3.10, fue necesario implementar varios componentes de firmware. Sin embargo, por ser los más relevantes, se mencionan los siguientes:

Un driver para el manejo de la pantalla *LCD RGB Backlight* de 16x2 que se llamó `lcd_i2c_grove.h`. Esta pantalla presenta la interfaz de comunicación I2C, para reducir la cantidad de pines utilizados por el microcontrolador. Este driver presenta las funciones descritas en el código 3.7. Este driver fue adaptado del driver proporcionado por el fabricante en: [11].

```

1  /*@brief Inicializa el bus I2C y las opciones de la LCD.
2   */
3  void lcd_init();
4
5  /* @brief Escribe en la LCD el texto deseado.
6   * @param row: Columna desde la cual se inicia la escritura.
7   * @param column: Fila sobre la cual se desea escribir.
8   * @param str: Texto que se desea escribir.
9   */
10 void lcd_write(uint8_t row, uint8_t column, char *str);
11
12 /* @brief Borra todos los caracteres de la LCD.
13  */
14 void lcd_clear();
15
16 /* @brief Cambia el color de fondo de la LCD.
17  * @param r: valor para cantidad de color rojo.
18  * @param g: valor para cantidad de color verde.
19  * @param b: valor para cantidad de color azul.
20  */
21 void lcd_set_RGB(unsigned char r, unsigned char g, unsigned char
           ↪ b);

```

CÓDIGO 3.7. Funciones principales del driver LCD.

Una función para inicializar los servicios MQTT en el módulo SIM A7670SA y conectarse al *broker* MQTT, que se llamó `init_sequence_mqtt_server()`. Esta

función implementa comandos AT específicos para el módulo en mención. Retorna el estado de conexión con el *broker* MQTT o en su defecto los errores encontrados en el proceso. En el código 3.8 se muestra el pseudocódigo que describe su funcionamiento. En la línea 3 del código, se envía un comando AT para iniciar el servicio MQTT en el dispositivo. La línea 4 espera la confirmación exitosa de la inicialización para proceder, en la línea 5, con la adquisición de un cliente MQTT en el dispositivo y se espera una respuesta exitosa (ver línea 6). Si se recibe "OK", se envía el comando para conectarse con el *broker* MQTT remoto (ver línea 7), si la conexión se establece, se retorna el estado de la conexión. De la misma manera, si alguno de los pasos anteriores falla, se retornan los errores correspondientes.

```

1  init_sequence_mqtt_server(uart1_event, at_response)
2  {
3      Enviar AT: iniciar el servicio MQTT
4      if ("OK" == respuesta)
5          Enviar AT: adquirir cliente MQTT
6          if ("OK" == respuesta)
7              Enviar AT: conectar con broker MQTT
8              if ("OK" == respuesta)
9                  return: MQTT_SERVER_OK
10             else
11                 return: MQTT_FAIL_INIT_SERVER
12         else
13             return: MQTT_FAIL_ADQ_CLIENT
14     else
15         return: MQTT_FAIL_INIT_SERVICE
16 }
```

CÓDIGO 3.8. Pseudocódigo de la función `init_sequence_mqtt_server()`.

La función `transmit_msg_mqtt()` para encapsular la funcionalidad de transmitir al *broker* MQTT una vez se ha establecido la conexión. Esta función implementa comandos AT específicos del módulo SIM A7670SA y retorna el estado de transmisión del mensaje. En el código 3.9 se muestra el pseudocódigo que describe su funcionamiento. Esta función recibe cuatro parámetros:

1. `mqtt_payload`: es el mensaje que se desea enviar a través de MQTT.
2. `topic`: es el tema o canal a través del cual el mensaje será publicado en el *broker* MQTT.
3. `uart1_event`: es un puntero para el evento relacionado con el UART.
4. `at_response`: es un puntero para almacenar la respuesta esperada del comando AT.

En la línea 3 Se envía el comando AT al dispositivo para configurar el tema (`topic`) en el cual se publicará el mensaje MQTT. Este comando le indica al dispositivo que debe prepararse para enviar un mensaje en un tema específico. Si se recibe una respuesta con "OK", entonces en la línea 5 se procede a cargar el `payload` en la memoria del módulo. Si el proceso es exitoso, entonces se procede a publicar el mensaje (ver línea 7). Nuevamente, si la publicación es correcta se

retorna el estado del proceso. Si alguno de los pasos anteriores falla, se retornan los respectivos errores encontrados.

```

1 transmit_msg_mqtt(mqtt_payload, topic, uart1_event, at_response)
2 {
3     Enviar AT: configurar topic
4     if ("OK" == respuesta)
5         Enviar AT: cargar payload
6         if ("OK" == respuesta)
7             Enviar AT: publicar
8             if ("OK" == respuesta)
9                 return: MQTT_MSG_OK
10            else
11                return: MQTT_MSG_FAIL
12        else
13            return: MQTT_MSG_FAIL
14    else
15        return: MQTT_TOPIC_FAIL
16 }
```

CÓDIGO 3.9. Pseudocódigo de la función `init_sequence_mqtt_server()`.

La función `nmea_rmc_parser_r()`, que permite extraer la información de las cadenas tipo GNRMC recibidas por el módulo Quectel L76. Esta es una función de tipo *thread-safe* para facilitar la multitarea. En el código 3.10 se muestra el pseudocódigo que describe su funcionamiento. Esta función recibe dos parámetros:

1. `nmeaString`: es la cadena de datos NMEA que contiene la información GNSS.
2. `gnssData`: es el puntero a una estructura que se llenará con los datos GNSS extraídos de la cadena NMEA.

La función valida en la línea 4 que la trama recibida comience con \$, el cual es el caracter de inicio de las tramas NMEA. Luego se divide la cadena NMEA en segmentos o *tokens* separados por comas, cada uno de los cuales contiene un fragmento específico de información. Luego, se recorre esta cadena y se identifican los *tokens* relevantes (ver líneas 6 a 11). Tal es el caso de "GNRMC", que permite validar que la trama NMEA contiene información de posicionamiento proveniente de señales dos constelaciones distintas. Los demás *tokens* permiten extraer la latitud y la longitud para la conformación de las coordenadas geográficas. En las líneas 10 y 11 se utiliza una función para convertir las coordenadas en formato Grados, Minutos y Segundos (DMS: del inglés *Degrees, Minutes, Seconds*) a coordenadas en Grados Decimales (DD: del inglés *Degrees Decimal*).

```

1 nmea_rmc_parser_r(nmeaString, gnssData)
2 {
3     Verificar trama inicia con "$"
4     if ("$" == token_0)
5         Dividir trama en tokens por ","
6         Verificar que el primer token sea "GNRMC"
7         if("GNRMC" == token_1)
8             time = token_2
9             if("A" == token_3)
10                 latitud = convertir_DMS_to_DD(
11                     ↪ token_4, token_5)
12                 longitud = convertir_DMS_to_DD(
13                     ↪ token_6, token_7)
14             else
15                 return: NMEA_NO_VALID
16                 ↪
17         else
18             return: NMEA_NO_RMC
19     else
20         return: NMEA_NO_VALID
21 }

```

CÓDIGO 3.10. Pseudocódigo de la función `nmea_rmc_parser_r()`.

3.2. Diseño e implementación de la interfaz web

En esta sección se presentan las consideraciones de diseño y la metodología de implementación de la interfaz web.

3.2.1. Diseño de la interfaz web

Para el diseño de la interfaz web, se tuvo en cuenta satisfacer la historia de usuario descrita en la planificación de este trabajo y que motiva al desarrollo de este componente:

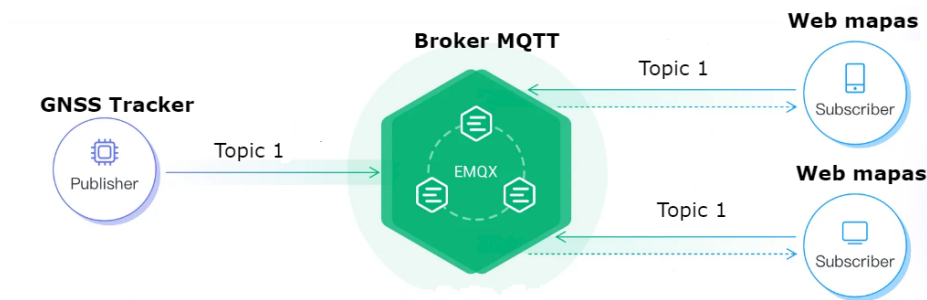
“Como usuario final quiero ver una página web con un mapa que me muestre la posición en tiempo real de todas las cavas y su estado de ocupación para realizar un seguimiento de estas y organizar la logística del reparto de productos en la cadena de frío”.

De acuerdo con lo anterior y con el análisis de la sección 2.3.5, se escogió la biblioteca *leaflet.js* [12] para la creación de mapas en la web, debido a que la empresa Cloud Technologys SAS, integra sus desarrollos web con tecnologías web con JavaScript y a que en la etapa de desarrollo del producto no se requerirá incurrir en gastos para el despliegue en pruebas o en entornos relevantes.

El desarrollo de la aplicación web se llevó a cabo siguiendo la arquitectura mostrada en la figura 3.14.

En esta arquitectura, el centro de la comunicación es el *broker* MQTT, donde se centralizan los mensajes de los *publishers* para ser transmitidos hacia los dispositivos *subscribers*, a través de canales específicos, llamados *topic*. El *broker* MQTT

¹Imagen adaptada de: <https://www.emqx.com/en/blog/mqtt-5-introduction-to-publish-subscribe-model>

FIGURA 3.14. Arquitectura MQTT¹.

es responsable de recibir conexiones iniciadas por el cliente y reenviar mensajes enviados por el cliente a otros clientes elegibles [13].

Para este trabajo, se utilizó el *broker* EMQX [14], que es utilizado para aplicaciones de IoT [13]. La aplicación web, al igual que el hardware desarrollado, son clientes del *broker* EMQX. El hardware es el encargado de publicar mensajes con los datos de geolocalización, estado de ocupación, validez de la posición, la hora y la referencia de la cava. Por otro lado, la aplicación web recibe dichos datos y los hace visibles al usuario a través de un mapa y otros elementos de visualización web.

3.2.2. Implementación de la interfaz web

Para la implementación de la interfaz web, se utilizaron las siguientes tecnologías:

- HTML: se utiliza para estructurar la página web.
- CSS (Bootstrap): para dar estilo a la página y facilitar la implementación, ya que cuenta con componentes predefinidos.
- Lenguaje de programación JavaScript, sus características y bibliotecas. A continuación, se detallan las más relevantes para este trabajo:
 - jQuery: biblioteca de JavaScript utilizada para facilitar la manipulación del DOM, eventos y AJAX.
 - Leaflet.js: biblioteca de JavaScript para mapas interactivos.
 - mqtt.js: biblioteca de JavaScript para conectarse a un *broker* MQTT, permite la comunicación en tiempo real entre el navegador y el *broker*.
- IDE Visual Studio Code para realizar la codificación.
- Docker como plataforma para la etapa de desarrollo, donde se ejecutó el EMQX.
- Broker EMQX.

3.2.3. Metodología de la implementación

A continuación, se presenta el proceso llevado a cabo para la implementación de la interfaz web.

1. Configuración inicial:

- Se diseñó la estructura de la página web usando HTML y Bootstrap. En este punto se utilizaron las bibliotecas necesarias para la funcionalidad del mapa y la conexión MQTT. En la figura 3.15 se observa la versión desarrollada por el autor. Además, en la figura 3.16 se muestra la versión implementada en el servidor de la empresa Cloud Technologys SAS, la cual fue migrada y ajustada por su equipo de desarrolladores web.
2. Configuración del mapa:
 - Con Leaflet.js se inicializa un mapa centrado en una posición predeterminada y se añade una capa de OpenStreetMap para mostrar el mapa base.
 3. Configuración de la conexión MQTT:
 - Se configuran las opciones de conexión al *broker MQTT*, a saber URL, credenciales de usuario, parámetro *keepalive* y el parámetro *connectTimeout*. Se establece la conexión con el *broker* utilizando *mqtt.js*. Por último, se realiza la suscripción al tópico que recibe los datos de ubicación del dispositivo GNSS.
 4. Gestión de mensajes:
 - Se define la función `get_cava_data()` para procesar los mensajes recibidos del *broker*. Se convierten los mensajes a formato JSON y se extraen los datos de la cava. Se actualiza el mapa con la nueva posición de la cava y se muestra la ruta seguida.
 5. Actualización del mapa:
 - Se elimina el marcador anterior y se añade un nuevo marcador en la nueva posición. Se actualiza la vista del mapa para centrarse en la nueva posición y se muestra la ruta recorrida por la cava utilizando una polilínea.

En el código 3.11 se muestra la implementación HTML de la interfaz web desarrollada por el autor, la cual sirvió de maqueta para que el equipo de desarrollo web de la empresa Cloud Technologys SAS lo implementara en sus servidores. Sin embargo, por petición del cliente, no es posible compartir el código implementado en sus servidores.

El código HTML inicia con la declaración del tipo de documento (`<!DOCTYPE html>`), ya que se trata de un archivo html5. Luego, se abre la etiqueta `<html>` que contiene toda la página web.

En la sección `<head>`, se define el título de la página como “Cava mapa”. También se incluyen enlaces a hojas de estilo y *scripts* externos. Específicamente, se carga *Bootstrap* desde un CDN (del inglés: *Content Delivery Network*), para estilos y componentes de interfaz de usuario, y *jQuery*, *Popper.js* y el propio *JavaScript* de *Bootstrap* para funcionalidad adicional. Además, se incluye una hoja de estilo para *Leaflet*.

En la sección `<body>`, se muestra un encabezado principal (`<h1>`) «GNSS Cava:», seguido de un encabezado secundario (`<h2>`) que especifica «Cava001».

Luego, hay una estructura de contenedor *Bootstrap* (<div class=container mt-4») que contiene una fila (<div class=row») con dos columnas (<div class=col-md-4» y <div class=col-md-8»). En la primera columna, se colocan varias tarjetas de información y un menú desplegable:

La primera tarjeta (<div class=card mb-3») tiene un cuerpo (<div class=card-body») que muestra el título «Dispositivo:» y un párrafo con el ID `cava_name` para mostrar el nombre del dispositivo.

La segunda tarjeta muestra el título «Posicion:» y dos párrafos con los IDs `lat` y `lng` para mostrar la latitud y longitud.

La tercera tarjeta muestra el título «Estado:» y un párrafo con el ID `estado` para mostrar el estado del dispositivo.

La segunda columna (que ocupa dos tercios del ancho) contiene un div (<div id=map») que servirá como contenedor para el mapa que se mostrará en la página.

Al final del cuerpo, se incluyen tres *scripts*: la biblioteca de *Leaflet*, la biblioteca MQTT y el archivo *JavaScript* específico de la aplicación `mapa.js`, que contiene la lógica para interactuar con el mapa y los datos de MQTT.

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cava mapa</title>
5
6     <link rel="stylesheet" href="https://stackpath.bootstrapcdn.
    ↪ com/bootstrap/4.5.2/css/bootstrap.min.css" crossorigin
    ↪ ="anonymous">
7     <script src="https://code.jquery.com/jquery-3.5.1.slim.min.
    ↪ js" crossorigin="anonymous"></script>
8     <script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2
    ↪ .5.3/dist/umd/popper.min.js" crossorigin="anonymous"
    ↪ ></script>
9     <script src="https://stackpath.bootstrapcdn.com/bootstrap
    ↪ /4.5.2/js/bootstrap.min.js" crossorigin="anonymous"></
    ↪ script>
10
11     <link rel="stylesheet" type="text/css" href="leaflet.css" />
12
13   </head>
14   <body>
15
16     <h1> GNSS Cava: </h1> <h2> Cava001 </h2>
17
18     <div class="container_mt-4">
19       <div class="row">
20         <div class="col-md-4">
21           <div class="card_mb-3">
22             <div class="card-body">
23               <h5 class="card-title">Dispositivo: </h5
    ↪ >
24               <p class="card-text" id="cava_name">h</p
    ↪ >
25             </div>
26           </div>

```

```

27         <div class="card_mb-3">
28             <div class="card-body">
29                 <h5 class="card-title">Posicion: </h5>
30                 <p class="card-text" id="lat"></p>
31                 <p class="card-text" id="lng"></p>
32             </div>
33         </div>
34         <div class="card_mb-3">
35             <div class="card-body">
36                 <h5 class="card-title">Estado: </h5>
37                 <p class="card-text" id="estado"></p>
38             </div>
39         </div>
40     </div>
41
42     <div class="col-md-8">
43         <div id="map"></div>
44     </div>
45
46 </div>
47 </div>
48
49 <script src="leaflet.js"></script>
50 <script src="mqtt.min.js"></script>
51 <script type="module" src="mapa.js"></script>
52 </body>
53
54 </html>

```

CÓDIGO 3.11. Código HTML de la página web.

GNSS Cava:

Cava001

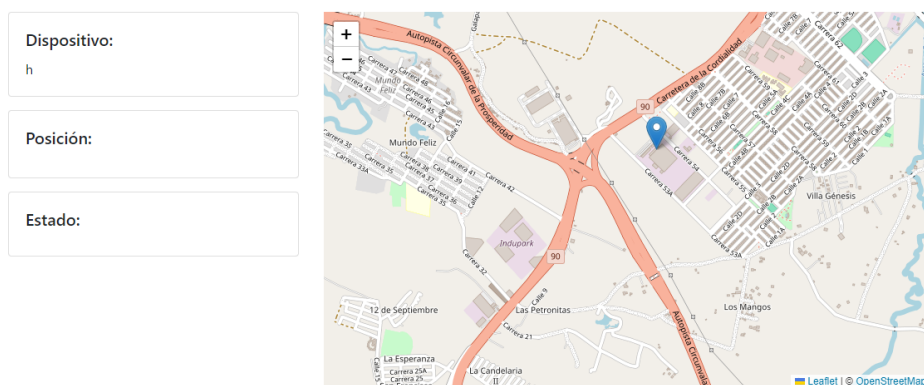


FIGURA 3.15. Maqueta de la interfaz web.

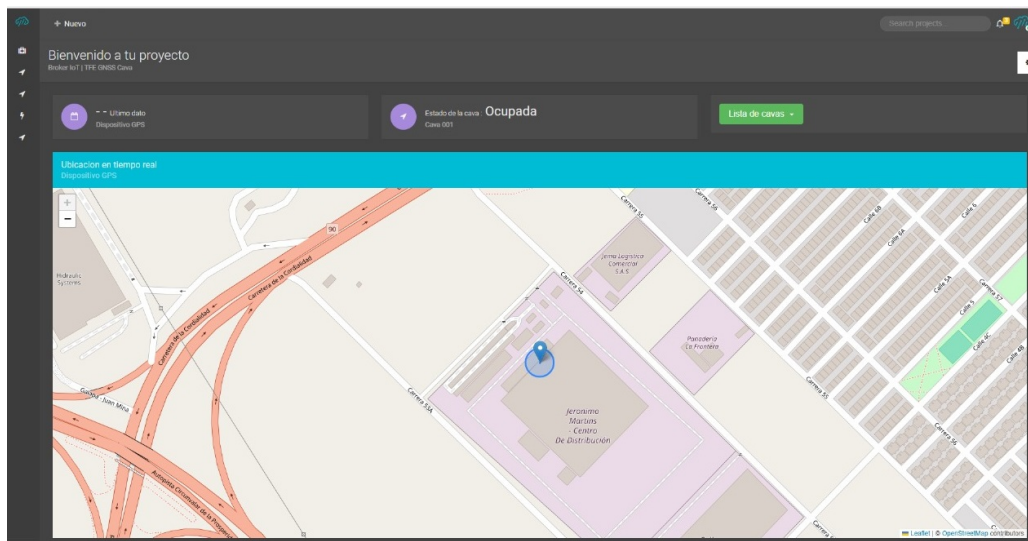


FIGURA 3.16. Versión final de la interfaz web.

En el código 3.12, se presenta la implementación de la dinámica de conexión del dispositivo y las funcionalidades necesarias para la inicialización del mapa y las conexiones.

El programa declara un mapa (`map`), un marcador (`marker`), una lista para almacenar el rastro de posiciones (`path`), y una variable para la línea que representará ese rastro (`polyline`). Además, define una posición inicial (`home`) con coordenadas específicas y una variable para la posición actual de la cava (`current_cava_position`).

A continuación, se configura una serie de opciones para conectarse al servidor MQTT, incluyendo el tiempo de espera para la conexión, el ID del cliente, el nombre de usuario, la contraseña y el parámetro *keepalive*. También se especifica la URL del servidor (`connectUrl`) y se crea un cliente MQTT con estas opciones. Se define un tema (`topic_cava`) al que la web se suscribe para recibir datos de la cava y se declara la variable (`cava_data`) para almacenar estos datos.

Se definen las siguientes funciones:

- La función `init_map` inicializa el mapa utilizando *OpenStreetMap*. Centra el mapa en la posición inicial (`home`) y añade un marcador en esta posición. Además, crea un `popup` para el marcador que muestra la latitud y longitud de la posición inicial.
- La función `get_cava_data` recibe datos de la cava, convierte las coordenadas de latitud y longitud de estos datos a números flotantes, y devuelve un objeto con estas coordenadas.
- La función `set_cava_position_map` se encarga de actualizar el mapa con la nueva posición de la cava. Primero, actualiza elementos HTML con los nuevos valores de latitud y longitud. Si ya existe un marcador, lo elimina del mapa. Luego, crea un nuevo marcador en la nueva posición y añade un `popup` con la información de esta posición. Además, añade la nueva posición a la lista `path` y dibuja un rastro en el mapa con esta lista. Finalmente, ajusta la vista del mapa para centrarlo en la nueva posición.

El programa llama a `init_map` para inicializar el mapa. Cuando el cliente MQTT se conecta, imprime un mensaje de “Conectado” y se suscribe al tema `topic_cava`. Maneja eventos de reconexión e impresión de errores en caso de que falle la conexión. Cuando el cliente recibe un mensaje, convierte el mensaje

a un objeto JSON, obtiene la posición de la cava utilizando `get_cava_data`, y llama a `set_cava_position_map` con esta posición para actualizar el mapa.

```

1  var map;
2  var marker;
3  let path = [];
4  let polyline
5  const home = { lat: 10.918982886682658, lng: -74.87194240611939
    ↪ };
6  let current_cava_position;
7
8  const options = {
9    clean: true,
10   connectTimeout: 4000,
11   clientId: '*****',
12   username: '*****',
13   password: '*****',
14   Keepalive: 60,
15 };
16
17 const connectUrl = 'ws://emqx*****';
18 const client = mqtt.connect(connectUrl, options);
19 const topic_cava = 'proyectoLuis/cava/datos';
20 let cava_data;
21
22 function init_map(){
23
24   map = L.map('map').setView([home.lat, home.lng], 13);
25   //OpenStreetMap
26   var tile_map = L.tileLayer('https://tile.openstreetmap.org/{
    ↪ z}/{x}/{y}.png', {
27     maxZoom: 19,
28     attribution: '&copy;_<a_href="http://www.openstreetmap.
    ↪ org/copyright">OpenStreetMap</a>'
29   });
30
31   tile_map.addTo(map)
32
33   marker = L.marker([home.lat, home.lng]).addTo(map);
34   marker.bindPopup("<b>Posicion</b>"+ "<br>"+ "lat:_"+home.lat.
    ↪ toString()+
35     "<br>"+ "_lng:_"+home.lng.toString()+
36     "<br>_CD"
37   ).openPopup();
38 }
39
40 function get_cava_data(cava_data) {
41
42   let latitude = parseFloat(cava_data.lat);
43   let longitude = parseFloat(cava_data.long);
44
45   return { lat: latitude, lng: longitude };
46 }
47
48 function set_cava_position_map(position){
49
50   document.getElementById('lat').textContent = "Lat:_"+
    ↪ position.lat.toString();

```

```

51     document.getElementById('lng').textContent = "Lng:␣"+
        ↪ position.lng.toString();
52
53     if (marker) {
54         map.removeLayer(marker);
55     }
56
57     marker = L.marker([position.lat, position.lng]).addTo(map);
58     var popup_info = "<b>Posicion</b>"+ "<br>"+position.lat.
        ↪ toString()+"␣"+position.lng.toString();
59     marker.bindPopup(popup_info).openPopup();
60
61     path.push([position.lat, position.lng]);
62
63     var rastro = L.polyline(path, {color: 'blue'}).addTo(map);
64
65     map.setView([position.lat, position.lng], 15);
66 }
67
68 init_map();
69
70 client.on('connect', function () {
71     console.log('Conectado')
72
73     client.subscribe(topic_cava, { qos: 0 }, function (err) {
74         if (!err) {
75             console.log('Suscrito')
76         }
77     })
78 });
79
80 client.on('reconnect', (error) => {
81     console.log('reconectando:', error)
82 });
83
84 client.on('error', (error) => {
85     console.log('Conexion_fallida:', error)
86 });
87
88 client.on('message', (topic, message) => {
89
90     current_cava_position = get_cava_data(JSON.parse(message.
        ↪ toString()));
91     set_cava_position_map(current_cava_position);
92 });

```

CÓDIGO 3.12. Implementación en javascript de la parte dinámica de la página web.

Capítulo 4

Ensayos y resultados

Este capítulo presenta los ensayos realizados para evaluar el desempeño del prototipo y los resultados obtenidos. Los ensayos se desarrollaron para validar la funcionalidad y exactitud del sistema en condiciones operativas de laboratorio.

4.1. Resultado final de la integración del hardware

En esta sección se presentan los resultados finales de la construcción del hardware. En la figura 4.1 se muestra una vista externa de la versión final del prototipo construido. En la figura 4.2 se revela una vista interna del prototipo, donde se puede ver el cableado y los módulos de hardware conectados a la placa base del sistema.



FIGURA 4.1. Resultado final del hardware desarrollado (exterior).

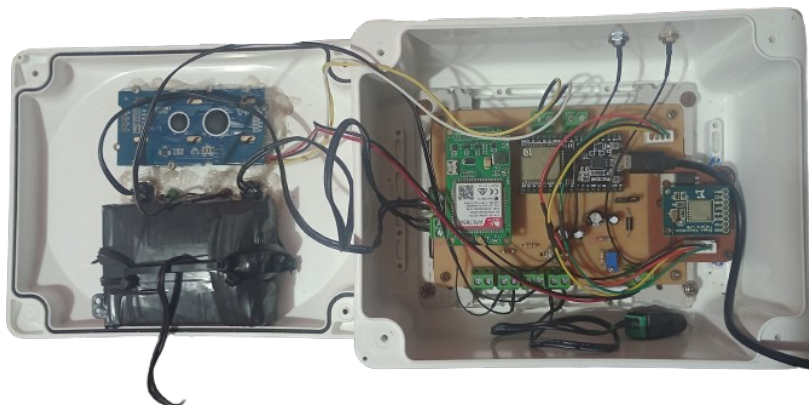


FIGURA 4.2. Resultado final del hardware desarrollado (interior).

En la figura 4.3 se señalan los siguientes componentes:

1. Pantalla LCD.
2. Botones y pilotos.

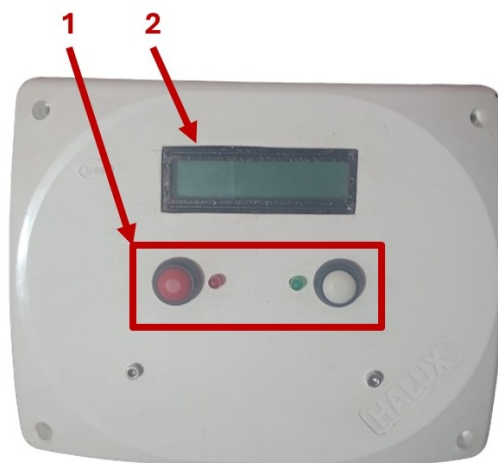


FIGURA 4.3. Vista real de los componentes del hardware (exterior).

En la figura 4.4 se señalan los siguientes componentes:

1. Batería.
2. Módulo SIM A7670SA.
3. ESP32.
4. Conectores SMA hembra para antena celular (izq.) y para antena GNSS (dcha.).
5. Módulo Quectel L76.

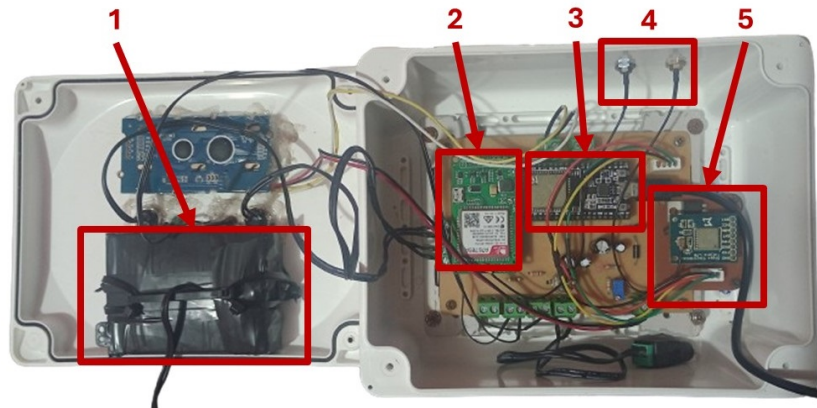


FIGURA 4.4. Vista real de los componentes del hardware (interior).

4.2. Pruebas funcionales del hardware

En esta sección se describe la metodología seguida para realizar las pruebas del hardware, se presentan los resultados que se obtuvieron y su correspondiente análisis.

4.2.1. Metodología para las pruebas del hardware

Se ejecutaron pruebas sobre los módulos de hardware para verificar que funcionan e interactúan adecuadamente entre sí. A continuación, se describe la metodología ejecutada:

1. En la PCB desarrollada:
 - a) Se verificaron las conexiones de los componentes para observar que no existen contactos abiertos, en cortocircuito o fallas a tierra.
 - b) Se verificó que a la salida del circuito de regulación se da un nivel de tensión de 5 VDC, necesarios para la alimentación de cada módulo.
 - c) Se midió el consumo de corriente de todo el hardware a plena carga, es decir, en el momento en que todas las tareas para las que fue diseñado están en ejecución.
2. Se realizaron pruebas unitarias de comunicación entre los diferentes módulos montados sobre la PCB. Para esto, se realizó lo siguiente:
 - a) Se probó la recepción de tramas NMEA con el módulo Quectel L76, a través del puerto UART. Se construyó un código de prueba que capturó las tramas NMEA recibidas por el puerto UART_1 y su eco en el puerto UART_0 para visualizar los datos recibidos en la computadora, a través de la terminal *Hercules* [15].
 - b) Se probó la comunicación bidireccional con el módulo SIM A7670SA, a través del puerto UART y por comandos AT. Se construyó un código de prueba que captura las tramas AT, recibidas por puerto UART_1 y su eco en el puerto UART_0 para visualizar los datos recibidos en la computadora, a través de la terminal *Hercules*. Este mismo programa transmite las tramas AT enviadas desde el puerto UART_0 hasta el módulo SIM A7670SA en el UART_1. De esta manera, se pueden enviar comandos AT de forma manual hacia el módulo y recibir las respuestas respectivas.

- c) Se realizó un código de prueba para utilizar todas las funciones proporcionadas por el driver `lcd_i2c_grove.h` y verificar la visualización correcta de textos predefinidos en la pantalla.

4.2.2. Resultados y análisis

De las pruebas sobre la PCB

Se verificó que no existen contactos abiertos, cortocircuitos o fallas a tierra antes y después del proceso de soldadura de la PCB. Se verificó la salida del circuito de regulación, y se observa una salida de 4.9 VDC suficiente para la alimentación de los módulos que componen el sistema, en la figura 4.5 se muestra una fotografía de la prueba.



FIGURA 4.5. Medición de la salida en el circuito de regulación.

La medición de la corriente total consumida por el hardware se llevó a cabo por un período de 30 segundos contados a partir de que se enciende el dispositivo; este es el tiempo que le toma al sistema inicializarse y comenzar a enviar datos al servidor. Resultó que existen variaciones desde los 130 mA hasta los 190 mA, además de valores con picos de 210 mA, de corta duración.

Se observó que los mayores picos de corriente se dan cuando el módulo SIM A7670SA recién se conecta a la red celular. También, se observa que el valor de corriente que más se repite en las mediciones es 150 mA. En la figura 4.6 se muestra el gráfico de las muestras tomadas; la secuencia de valores corresponde temporalmente, es decir, que describe el comportamiento de la corriente en el tiempo. Además, en la figura 4.7 se muestran fotografías de las mediciones más representativas.

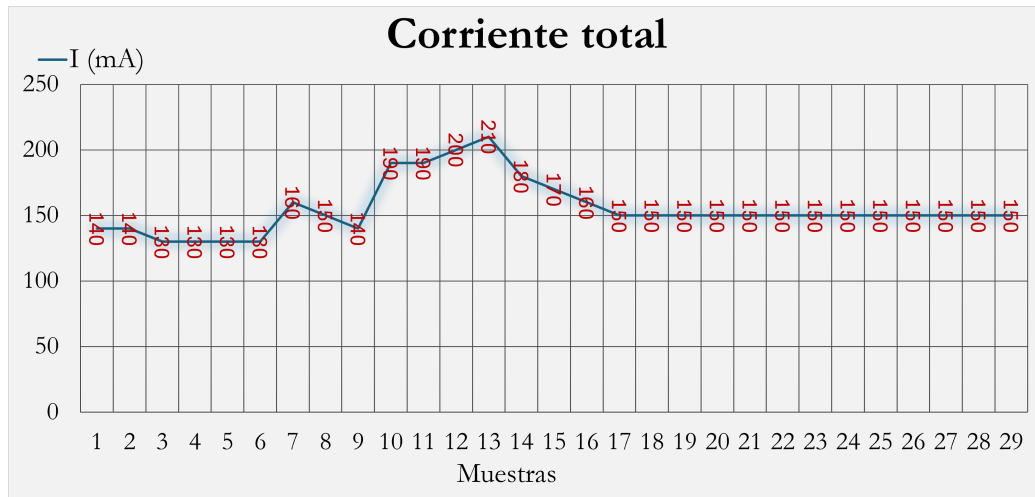


FIGURA 4.6. Gráfico de la muestras tomadas en la medición de corriente.



FIGURA 4.7. Datos más representativos en la medición de corriente.

De las pruebas entre módulos

Prueba del módulo Quectel L76

En la figura 4.8 se muestra un diagrama de la conexión necesaria para esta prueba. En la figura 4.9 se muestra la conexión real realizada. En esta configuración se realiza lo siguiente:

1. Para esta prueba se requiere el uso del puerto UART_1, por lo tanto, se intercambian los pines GND, Vin, Tx y Rx del módulo Quectel L76 en la

ranura del módulo SIM A7670SA. Debido a que son conectores diferentes se deben emplear cables de prueba.

2. Se deben desconectar los módulos SIM A7670SA y Quectel L76 de sus respectivas posiciones designadas, para no interferir la comunicación serial.

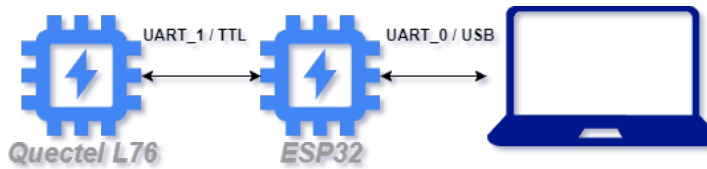


FIGURA 4.8. Diagrama de conexiones para la prueba del módulo Quectel L76.

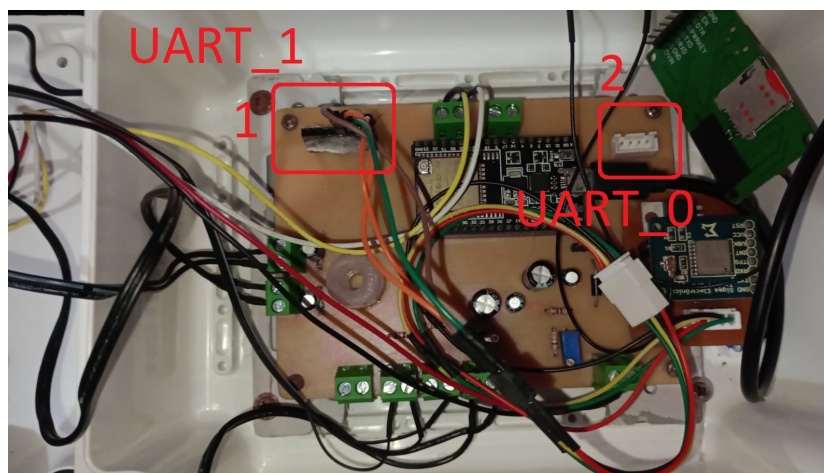


FIGURA 4.9. Conexiones reales para la prueba del módulo Quectel L76.

El código de prueba implementado es el que se muestra en el código 4.1. Por simplicidad y reducir el espacio en este documento, se han obviado las declaraciones de variables, macros y la función `main()`. Este programa utiliza *FreeRTOS* para crear dos tareas, la primera `uart1_interrupt_task()`, que se encarga de recibir las tramas del puerto UART_1 (donde está conectado el módulo L76) y retransmitirlas al computador por el puerto UART_0; y la segunda `uart0_interrupt_task()` permite enviar comandos PMTK desde el computador hacia el módulo L76. De esta manera se comprueba que hay una comunicación adecuada entre los módulos. La función `main()` se encarga de inicializar los puertos UART en modo de interrupciones y crear las tareas correspondientes.

```

1 static void uart1_interrupt_task(void *params)
2 {
3     uart_event_t uart_event;
4     uint8_t *uart_rcv_data = (uint8_t *)malloc(BUF_SIZE);
5     while (1)
6     {
7         if (xQueueReceive(uart1_queue, &uart_event,
8             ↪ portMAX_DELAY))
9         {
10             bzero(uart_rcv_data, BUF_SIZE);
11             switch (uart_event.type)
12             {
13                 case UART_DATA:
14                     uart_receive(UART1, uart_rcv_data, uart_event.
15                         ↪ size);
16                     uart_transmit(UART0, uart_rcv_data, strlen(
17                         ↪ uart_rcv_data));
18                     bzero(uart_rcv_data, BUF_SIZE);
19                     break;
20             }
21         }
22     }
23     free(uart_rcv_data);
24 }
25 static void uart0_interrupt_task(void *params)
26 {
27     uart_event_t uart_event;
28     uint8_t *uart_rcv_data = (uint8_t *)malloc(BUF_SIZE);
29     while (1)
30     {
31         if (xQueueReceive(uart0_queue, &uart_event,
32             ↪ portMAX_DELAY))
33         {
34             bzero(uart_rcv_data, BUF_SIZE);
35
36             switch (uart_event.type)
37             {
38                 case UART_DATA:
39                     uart_receive(UART0, uart_rcv_data, uart_event.
40                         ↪ size);
41                     uart_transmit(UART1, uart_rcv_data, strlen(
42                         ↪ uart_rcv_data));
43                     break;
44             }
45         }
46     }
47     free(uart_rcv_data);
48 }

```

CÓDIGO 4.1. Código para probar la comunicación con el módulo Quectel L76.

Luego de implementar las configuraciones anteriores, se obtuvieron las tramas NMEA proporcionadas por el módulo Quectel L76. La prueba resultó exitosa, ya que es posible capturar dichas tramas y procesarlas. En la figura 4.10 se muestra

una captura de las tramas visualizadas en la terminal *Hercules*. En esta captura se puede observar las tramas que incian con \$GNRMC, las cuales contienen la información de geolocalización.

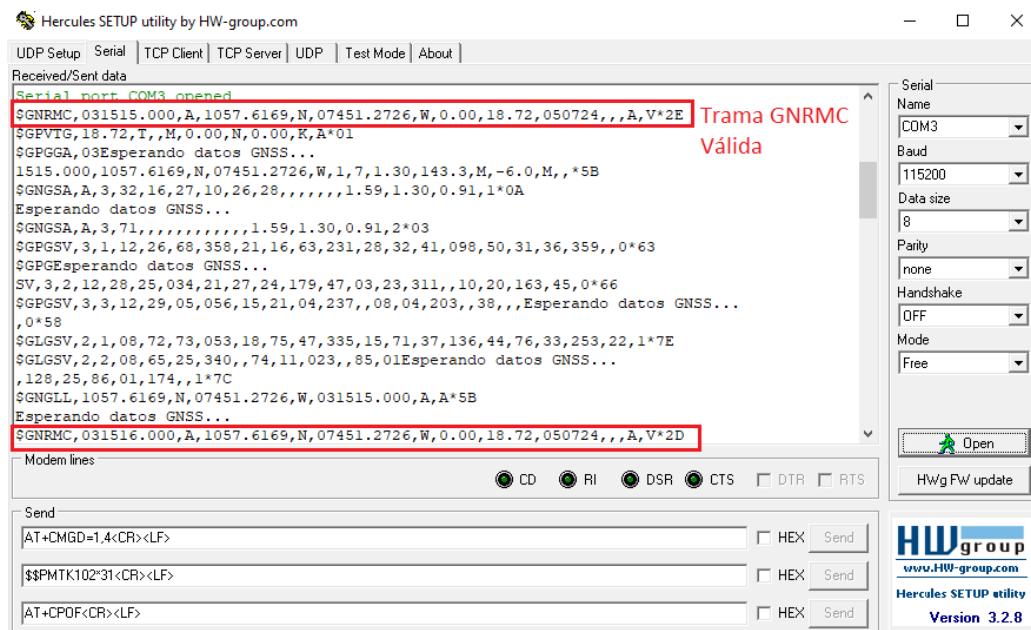


FIGURA 4.10. Vista de las tramas NMEA en la terminal *Hercules*.

Prueba del módulo SIM A7670SA

En la figura 4.11 se muestra un diagrama de la conexión necesaria para esta prueba. En la figura 4.12 se muestra la conexión real realizada. En esta configuración se realiza lo siguiente:

1. Se mantiene conectado el módulo SIM A7670SA ya que tiene designado el puerto UART_1.
2. Se desconecta el módulo Quectel L76 de su ranura, para que no interfiera la comunicación serial por el puerto UART_0.

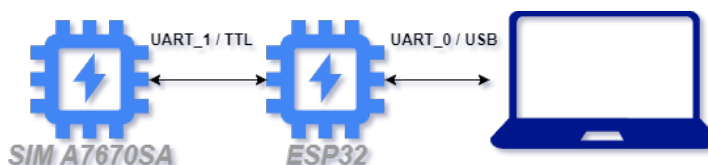


FIGURA 4.11. Diagrama de conexiones para la prueba del módulo Quectel L76.

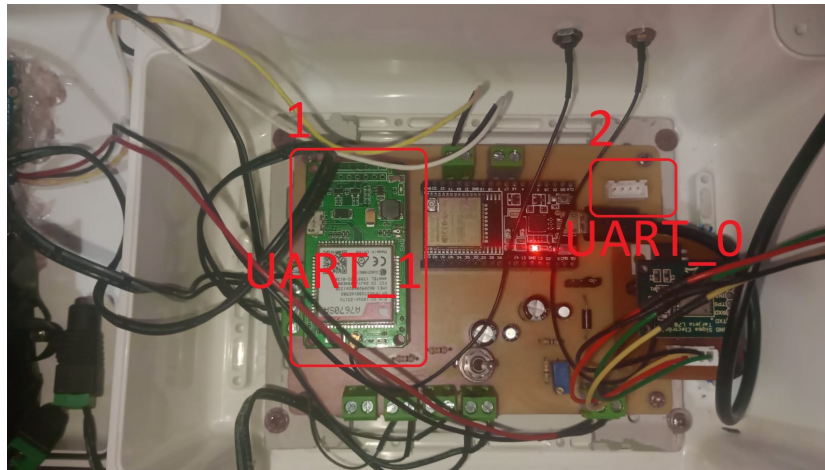
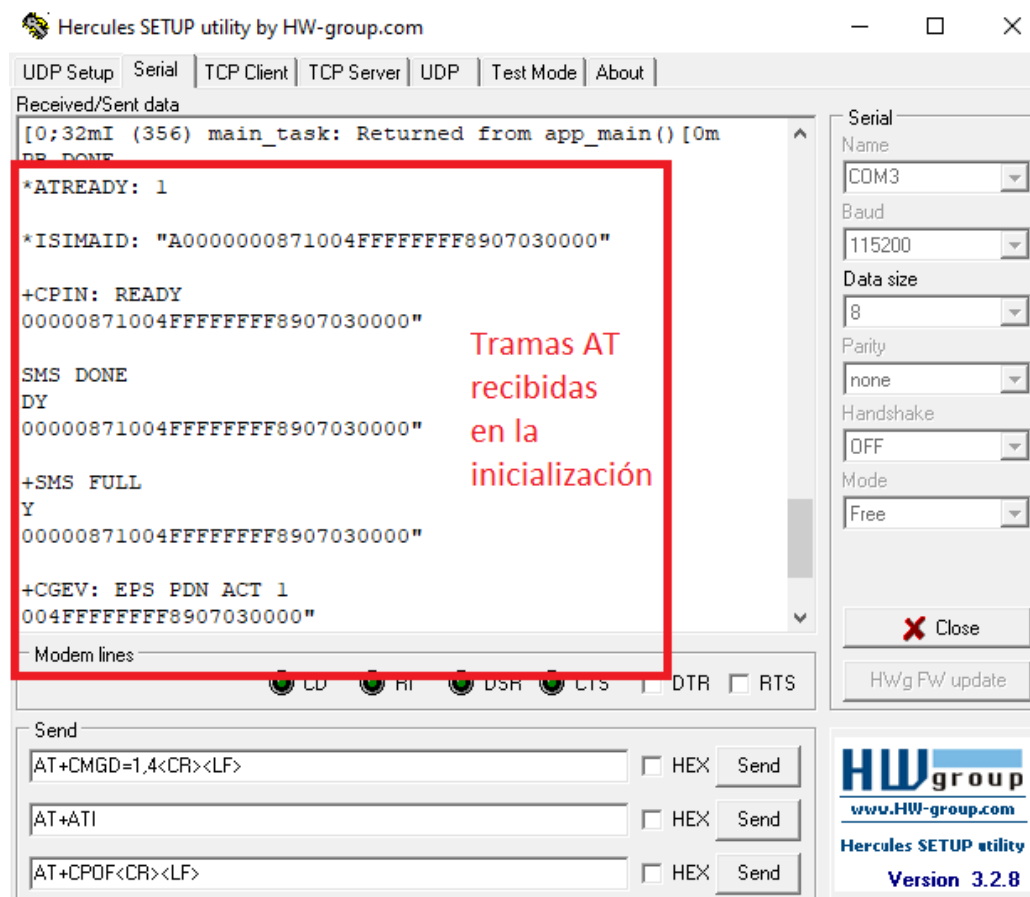


FIGURA 4.12. Conexiones reales para la prueba del módulo Quectel L76.

El código de prueba implementado es el mismo que se implementó en la prueba anterior (ver código 4.1). Igual que en la prueba anterior, la tarea `uart1_interrupt_task()`, se encarga de recibir las tramas del puerto UART_1 (donde está conectado el módulo SIM A7670SA) y retransmitirlas al computador por el puerto UART_0; y la tarea `uart0_interrupt_task()` permite enviar comandos AT desde el computador hacia el módulo SIM A7670SA, de forma manual desde la terminal *Hercules*.

Luego de implementar las configuraciones anteriores, se obtuvieron las tramas AT que responde el módulo SIM A7670SA y se pudieron transmitir comandos AT hacia este módulo. La prueba resultó exitosa, ya que es posible capturar dichas tramas y procesarlas.

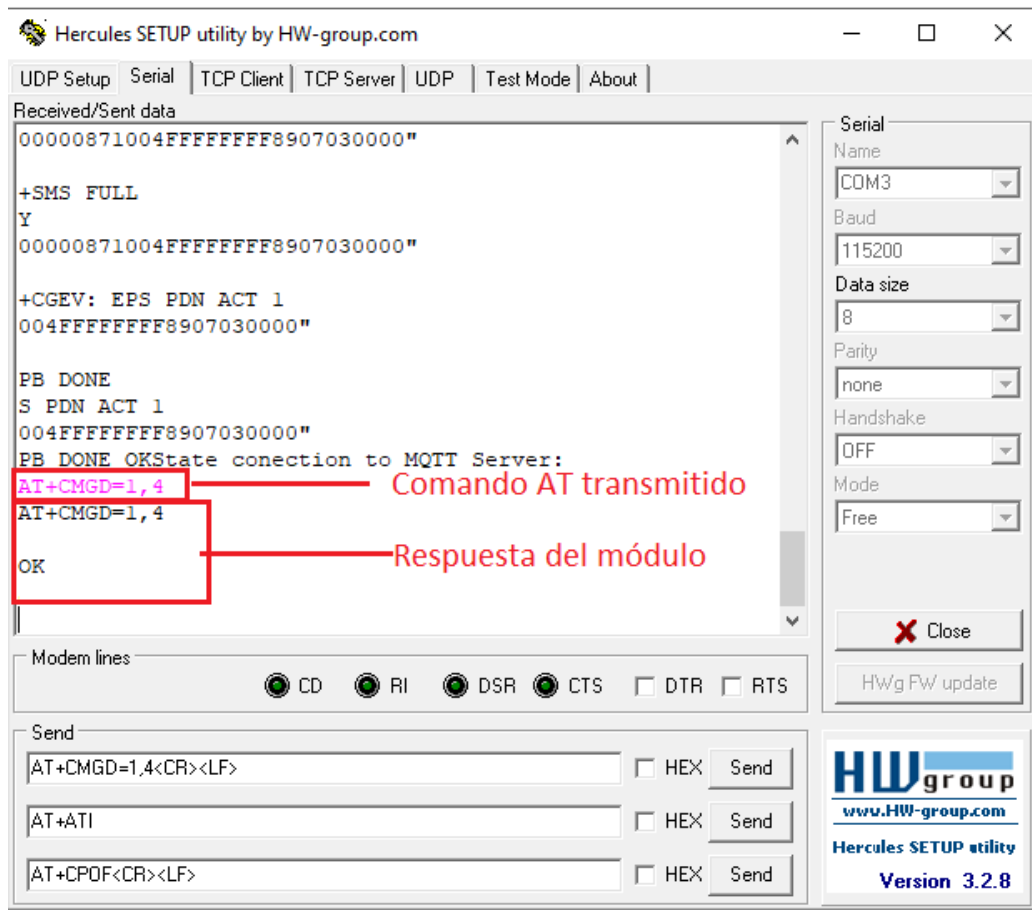
En la figura 4.13 se muestra una captura de los mensajes visualizados en la terminal *Hercules*. En esta captura se puede observar mensajes recibidos del módulo SIM A7670SA que inician los símbolos + y *, los cuales contienen información sobre los procesos del módulo.

FIGURA 4.13. Vista de las tramas AT en la terminal *Hercules*.

En la figura 4.14 se muestra otra captura de mensajes visualizados en la terminal *Hercules*. En esta captura se muestra la transmisión del comando AT+CMGD=1, 4 hacia el módulo SIM A7670SA, este comando debe cumplir con las siguientes condiciones:

1. Debe iniciar con los caracteres AT+.
2. Comando a enviar. En el caso de la prueba realizada, se envió el comando CMGD que se usa para eliminar mensajes SMS.
3. Los dos parámetros del comando: en este caso <index>(1) y <delflag>(4), se utilizan para borrar todos los mensajes en la tarjeta SIM.
4. Por último, los símbolos de retorno de carro <CR> y salto de línea <LF>. Si estos dos símbolos no están presentes, el comando no será reconocido.

En la figura 4.14 también se observa la respuesta del módulo al comando enviado. El módulo responde con un texto que contiene el comando y "OK" para indicar que fue ejecutado con éxito.

FIGURA 4.14. Vista de las tramas AT en la terminal *Hercules*.

Prueba de la pantalla LCD

Para esta prueba no es necesaria ninguna configuración especial, más allá de las conexiones de diseño del hardware.

Se implementó un código de prueba (ver código 4.2) que permitió usar las funciones proporcionadas por el driver `lcd_i2c_grove.h`. Este sencillo programa escribe textos y cambia los colores de la pantalla LCD cada tres segundos. Además, borra la pantalla cada vez que escribe un texto nuevo. De esta manera se puede corroborar el funcionamiento de la LCD. En la figura 4.15 se muestran fotografías de los resultados de la prueba.

```
1 void app_main()
2 {
3     lcd_init();
4     lcd_clear(); lcd_set_RGB(255, 255, 255); //pantalla blanca
5     lcd_write(0, 0, "Linea_0"); lcd_write(1, 0, "Linea_1");
6     delay(DELAY_LCD);
7
8     while(1)
9     {
10         lcd_clear();
11         lcd_write(0, 0, "Color:_"); lcd_write(1, 0, "Rojo");
12         lcd_set_RGB(255, 0, 0); //pantalla roja
13         delay(DELAY_LCD);
14
15         lcd_clear();
16         lcd_write(0, 0, "Color:_"); lcd_write(1, 0, "Verde");
17         lcd_set_RGB(0, 255, 0); //pantalla verde
18         delay(DELAY_LCD);
19
20         lcd_clear();
21         lcd_write(0, 0, "Color:_"); lcd_write(1, 0, "Azul");
22         lcd_set_RGB(0, 0, 255); //pantalla azul
23         delay(DELAY_LCD);
24
25         lcd_clear();
26         lcd_write(0, 0, "Color:_"); lcd_write(1, 0, "amarillo");
27         lcd_set_RGB(255, 255, 0); //pantalla amarilla
28         delay(DELAY_LCD);
29
30         lcd_clear();
31         lcd_write(0, 0, "Color:_"); lcd_write(1, 0, "magenta");
32         lcd_set_RGB(255, 0, 255); //pantalla magenta
33         delay(DELAY_LCD);
34
35         lcd_clear();
36         lcd_write(0, 0, "Color:_"); lcd_write(1, 0, "Cyan");
37         lcd_set_RGB(0, 255, 255); //pantalla cyan
38         delay(DELAY_LCD);
39     }
40 }
```

CÓDIGO 4.2. Código para probar comunicación con la pantalla LCD.

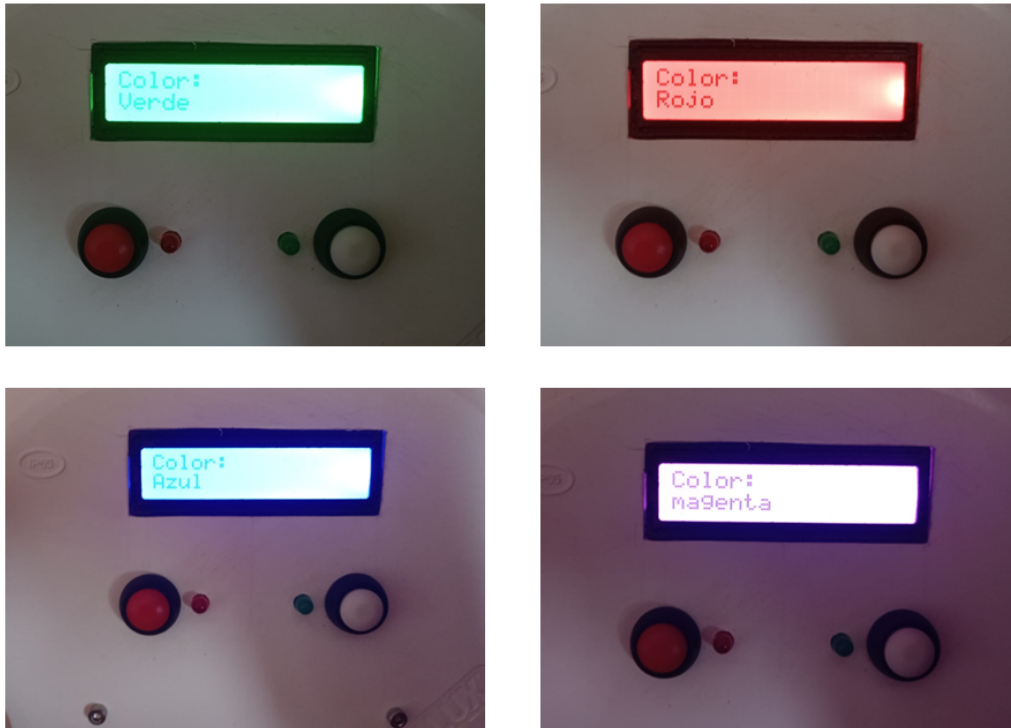


FIGURA 4.15. Fotos de la prueba de la pantalla LCD.

4.3. Pruebas funcionales de la aplicación web

En esta sección se describe la metodología seguida para realizar las pruebas de la aplicación web, se presentan los resultados que se obtuvieron y su correspondiente análisis.

Las pruebas llevadas a cabo fueron las siguientes:

1. Prueba de integración con el hardware.
2. Prueba de exactitud de la ubicación.

4.3.1. Prueba de integración con el hardware

En este escenario se probó la capacidad de la aplicación web para conectarse con el *broker* EMQX y procesar la información que transmite el hardware. Para esto, se ejecutaron los siguientes pasos:

Metodología

1. Se puso en funcionamiento normal el hardware.
2. Se accedió al enlace donde se aloja la página web del servicio de mapas.
3. Se comprobó la recepción de la ubicación actual en una trayectoria de 100 m.

Resultados y análisis

En la figura 4.16 se muestra la aplicación de web con el mapa y la trayectoria descrita por el movimiento del hardware. En esta prueba el recorrido se hizo a pie debido a la corta distancia a recorrer. Se obtuvo una respuesta adecuada de

la interacción del sistema completo, desde la recepción de la geolocalización, la transmisión de la información al *broker* a través de internet en la red celular, hasta la visualización en la web.

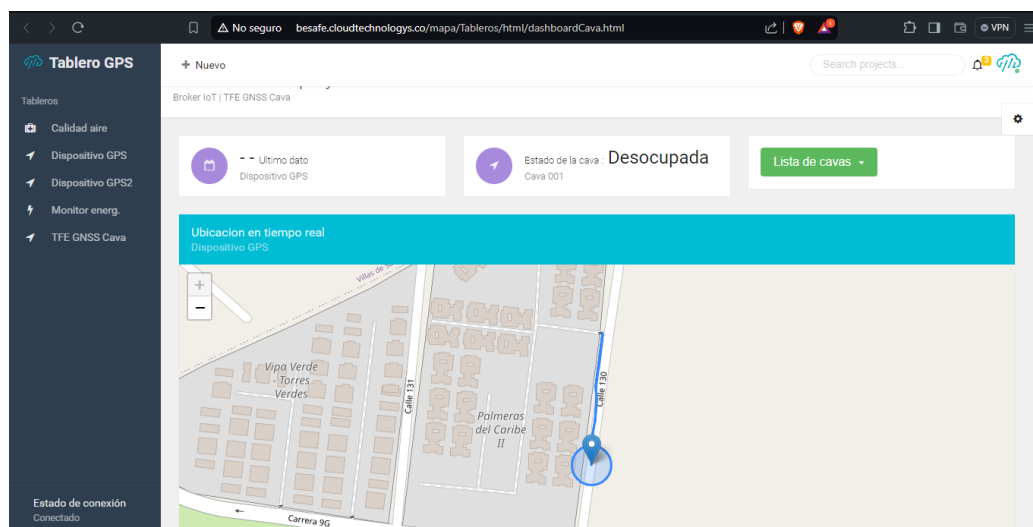


FIGURA 4.16. Captura del mapa para la prueba de integración con el hardware.

4.3.2. Prueba de exactitud de la ubicación

Para realizar esta prueba, se procedió a verificar la exactitud de la localización que proporciona el hardware. Para esto, se capturaron los datos de posición en diversos escenarios y se compararon las coordenadas obtenidas con las coordenadas reales conocidas. A continuación, se describe la metodología llevada a cabo:

Metodología

1. Se seleccionaron posiciones en puntos fijos en la ciudad de Barranquilla. En un conjunto de edificios residenciales con calles internas para recrear espacios urbanos con presencia de edificios.
2. Se marcaron las coordenadas geográficas de los puntos seleccionados. Para esto se utilizó la aplicación *Google Map* y un teléfono inteligente como instrumento patrón.
3. En cada punto de prueba se registraron las coordenadas proporcionadas por el prototipo y por el teléfono inteligente. Se tomaron cinco muestras con una separación de un minuto. El valor de la medición que se seleccionó fue el que más se repite de las cinco muestras.
4. Se calculó la distancia entre las coordenadas registradas por el prototipo y las coordenadas tomadas de referencia de *Google Map*. El cálculo de esta distancia se realizó con la ecuación de *Haversine* [16]. El cálculo se toma como el grado de exactitud en la geolocalización realizada por el prototipo, al considerar como valor verdadero el proporcionado por el teléfono inteligente.

Resultados y análisis

Luego de realizadas las pruebas se tabularon los datos obtenidos (ver tabla 4.1).

Las muestras 1 a 8 de la tabla 4.1, corresponden a posiciones en las calles, con edificios al rededor. En estos registros se observa que la exactitud está entre 0 y 5 m.

Por otro lado, las muestras 9 a 11 de la tabla 4.1 corresponden a coordenadas medidas dentro de los edificios, en los pasillos. Como era de esperar, el hormigón armado ofrece interferencia a la señal satelital, por lo tanto la exactitud de la medición baja considerablemente a alrededor de 40 m.

TABLA 4.1. Mediciones de la exactitud en la geolocalización

#	Prototipo		Google Maps		Distancia (m)
	Latitud 1	Longitud 1	Latitud 2	Longitud 2	
1	10,9605154	-74,8544642	10.96046963	-74,85447104	5,14
2	10,9609898	-74,8545152	10.96096967	-74,85449264	3,33
3	10,9605312	-74,8547626	10.96052396	-74,85474651	1,93
4	10,9605819	-74,8551536	10.96056399	-74,85516585	2,40
5	10,9601415	-74,8546094	10.96013613	-74,8545933	1,86
6	10,9598841	-74,8546406	10.95987377	-74,85462887	1,72
7	10,9593663	-74,8547036	10.95935576	-74,85469958	1,25
8	10,9594072	-74,8550654	10.95941637	-74,85509867	3,77
9	10,9605637	-74,8546886	10.96084288	-74,85487636	37,2
10	10,9602758	-74,854543	10.96033852	-74,85468831	17,3
11	10,9598994	-74,8546695	10.95995456	-74,85446811	22,8

Capítulo 5

Conclusiones

En esta sección se resaltan los principales aportes del trabajo realizado y cuáles podrían ser los trabajos futuros inspirados en este.

5.1. Conclusiones generales

A continuación, se presentan las conclusiones más relevantes:

- El presente trabajo dio como resultado un prototipo funcional de geolocalización basado en tecnología GNSS, que es capaz de transmitir la posición de una cava móvil hacia un *Broker* MQTT, que a su vez cuenta con una página web donde se puede visualizar en un mapa la posición actual.
- Los requerimientos del trabajo se cumplieron en un 95 % debido a que no se pudo cumplir con la exactitud de 10 m en el interior de edificaciones. Esta exactitud se cumple mientras el dispositivo se encuentra en la calle, lugares despejados o cerca de ventanas; sin embargo, cuando el dispositivo se encuentra al interior de un edificio de hormigón la exactitud baja considerablemente al rededor de 40 m.
- Según la planificación del trabajo, se esperaba haberlo terminado en el mes de mayo de 2024. Sin embargo, por situaciones relacionadas con obligaciones laborales del autor y retrasos debido a implementaciones fallidas del hardware no se pudo cumplir con los tiempos planificados.
- Se manifestó el riesgo 1 “daño o deterioro inesperado de los módulos de hardware”. Las medidas de mitigación fueron efectivas. Específicamente se tuvo que cambiar el módulo de comunicación GPRS/GSM Quectel M95, que estaba previsto en la planificación, por el módulo 4G SIM A7670SA. Esto debido a los cambios en la infraestructura de comunicaciones en Colombia, donde se discontinuó el uso de comunicaciones 2G. Lo anterior causó que se desechara el trabajo adelantado con el módulo M95 e impactó con retrasos en el trabajo.
- Debido a lo anterior, también se manifestó el riesgo 4 “trabajos no planificados por fallas en el análisis”, no se consideró correctamente la tecnología de telecomunicaciones a utilizar.
- Se manifestó el riesgo 5 “extensión del proyecto más allá del plazo establecido”. El plan de mitigación no fue efectivo, debido a variables incontrolables como: nueva carga laboral y otros imprevistos. Habría sido prudente analizar mejor la situación con respecto a los requerimientos para ajustar el alcance del trabajo.
- Se debió cambiar el módulo Quectel M95 por el SIM A7670. Esto produjo retrasos en el proyecto, pero trajo el beneficio de una actualización tecnológica importante que se ve reflejada en la estabilidad de la comunicación celular.

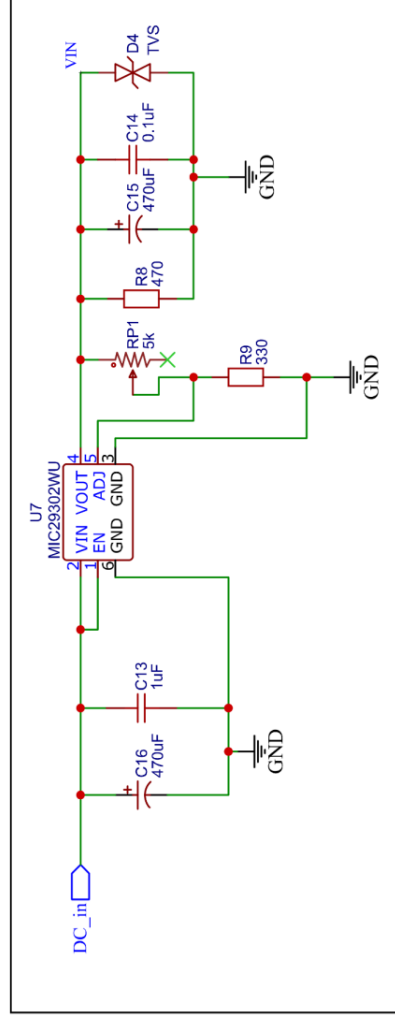
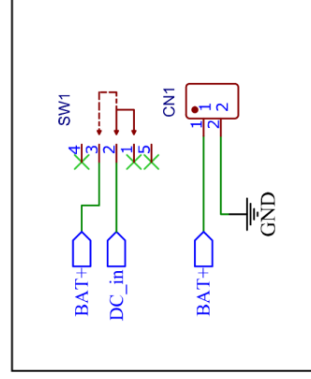
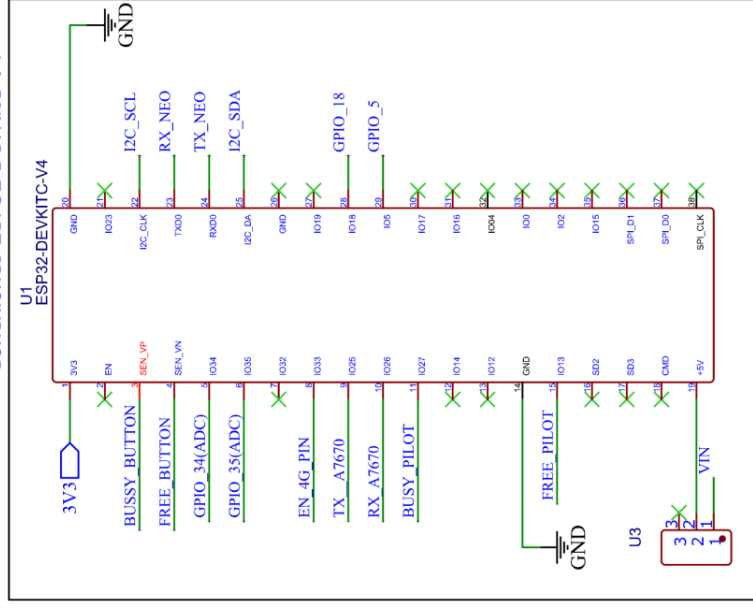
5.2. Próximos pasos

Como trabajo futuro quedan los siguientes:

- Cambiar el módulo 4G por otro que permita funciones de triangulación.
- Implementar un protocolo de seguridad de acceso a personal autorizado con tecnología RFID o huella digital.
- Realizar un diseño de hardware con miras a disminuir el consumo de potencia, optimizar conexiones y reducir el tamaño del dispositivo.
- Implementar una base de datos en la nube para registrar los históricos de posiciones a largo plazo.
- Desarrollar más funcionalidades a la página web como: selección de dispositivos, mostrar históricos de posiciones y estados, entre otros.

Apéndice A

Esquemático de la placa base del sistema



Bibliografía

- [1] *Aplicaciones GNSS*. <https://www.juntadeandalucia.es/institutodeestadisticaycartografia/rap/aplicaciones-gnss>. Accessed: 2023-11-01.
- [2] Oficina de Asuntos del Espacio Ultraterrestre Naciones Unidas. *Programa de estudios Sistemas mundiales de navegación por satélite*. ST/SPACE/59. 2013.
- [3] Domenico Di Grazia et al. «A Triple-Band GNSS Receiver for High Accuracy Automotive Applications». En: (2022), págs. 41-46. DOI: [10.1109/MetroAutomotive54295.2022.9855084](https://doi.org/10.1109/MetroAutomotive54295.2022.9855084).
- [4] Yuanming Shu et al. «High-Rate Attitude Determination of Moving Vehicles With GNSS: GPS, BDS, GLONASS, and Galileo». En: *IEEE Transactions on Instrumentation and Measurement* 71 (2022), págs. 1-13. DOI: [10.1109/TIM.2022.3168896](https://doi.org/10.1109/TIM.2022.3168896).
- [5] Christophe Villien y Benoît Denis. «UWB-aided GNSS/INS Fusion for Resilient Positioning in GNSS Challenged Environments». En: (2023), págs. 167-178. DOI: [10.1109/PLANS53410.2023.10139961](https://doi.org/10.1109/PLANS53410.2023.10139961).
- [6] *Hardware Reference - ESPRESSIF*. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/index.html>. Accessed: 2023-11-6.
- [7] *Receptor GNSS Quectel L76*. <https://www.quectel.com/product/gnss-l76>. Accessed: 2023-11-6.
- [8] *Módulo 4G SIMA7670SA*. <https://www.microchip.com/en-us/product/mic29302>. Accessed: 2023-11-6.
- [9] *The FreeRTOS™ Kernel*. <https://www.freertos.org/RTOS.html>. Accessed: 2023-11-6.
- [10] *Regulador MIC29302WU*. <https://www.simcom.com/product/A7670X.html>. Accessed: 2024-07-2.
- [11] *Pantalla LCD Grove*. https://github.com/Seeed-Studio/Grove_LCD_RGB_Backlight. Accessed: 2023-11-6.
- [12] *Leaflet.js: Overview*. <https://leafletjs.com/>. Accessed: 2024-07-11.
- [13] EMQ Technologies Inc. *Mastering MQTT. Your Ultimate Tutorial for MQTT*. R24-3. 2024.
- [14] *EMQX*. <https://www.emqx.io/>. Accessed: 2024-07-11.
- [15] *Hercules SETUP utility*. <https://www.hw-group.com/software/hercules-setup-utility>. Accessed: 2024-07-11.
- [16] Huseyin Alkan y Hasari Celebi. «The Implementation of Positioning System with Trilateration of Haversine Distance». En: *2019 IEEE 30th Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*. 2019, págs. 1-6. DOI: [10.1109/PIMRC.2019.8904289](https://doi.org/10.1109/PIMRC.2019.8904289).